Name: P. Lakshmi Gayathri Navaneetha
Reg no: 192321169

CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 3 – CO3

CO3 Coverage: Logical agents – Propositional logic – Reasoning patterns in propositional logic – Resolution – First-order logic – Knowledge engineering in FOL – Inference in first-order logic – Propositional versus first-order inference – Unification and lifting – Forward chaining – Backward chaining – Resolution.

Mode: Pair Work – 2 Students per Group
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Design → Implementation → Analysis → Evaluation.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 Marks) Part A – Knowledge Base and Model Checking in the Wumpus World

Shared Group Work: Together encode the rules of an $n \times n$ Wumpus world in propositional logic (pit, wumpus, breeze and stench symbols for every square and the rules relating percepts to neighbouring hazards). Implement a truth-table entailment checker and query it with percept sequences that you design, for example, "Is square [2,2] safe?".

Student A – Design and Mathematical Analysis

- Count the proposition symbols and sentences as a function of n
- Derive the number of models 2^m and identify the minimal set of symbols relevant to one query • Explain what soundness and completeness mean for model checking

Student B – Experimental Validation

- Instrument the checker to record models enumerated, KB evaluations and time • Run it for n = 2, 3, 4 (state where the full symbol set becomes infeasible) and again with relevant symbol restriction
- Include one entailed query, one query whose negation is entailed and one query for which neither is entailed

Part B – Reasoning Patterns

Shared Group Work: Using the same Wumpus-world KB, derive the safety of at least two squares by applying inference rules (Modus Ponens, And-Elimination, Unit Resolution) instead of model checking.

Student A – Experimental Investigation: Implement the inference-rule derivation, record the number of inference steps and compare with the model-checking cost for the same query.

Student B – Soundness Reasoning: Prove the soundness of each rule used with a truth table and explain why a derived sentence must hold in every model of the KB, and why "not derived" is not the same as "false".

Cross-Understanding Requirement: Each student must be able to explain the other student's component.

Evidence Expected: KB listing, symbol-count derivation, checker code and outputs for the three query types, inference-rule derivations with step counts, soundness tables and a comparison of the two reasoning approaches.

ANSWER – TASK 1: Knowledge Bases, Entailment and Reasoning Patterns

All experiments in this answer were produced by the accompanying Python code (`prop.py`, `t1_wumpus.py`); the raw outputs are in `data/t1.json`. Timings are from a single sandbox machine and are indicative, while the counts (models, evaluations, steps) are exact and machine independent.

Shared group work – propositional encoding of the n × n Wumpus world

For every square [x,y] there are four symbols: **P**xy (pit), **W**xy (wumpus), **B**xy (breeze) and **S**xy (stench). The KB contains four groups of sentences:

- **Start square** (2): ¬P11 and ¬W11.
- **Percept rules** ($2n^2$): Bxy ⇔ (disjunction of Pa,b over the neighbours of [x,y]) and Sxy ⇔ (disjunction of Wa,b over the neighbours).
- **At least one wumpus** (1): W11 ∨ W12 ∨ … ∨ Wnn.
- **At most one wumpus** ($n^2(n^2-1)/2$): ¬Wa ∨ ¬Wb for every unordered pair of squares.

Complete KB listing for n = 2 (17 sentences; neighbours of [1,1] are [1,2] and [2,1], and so on). Percept facts are added separately as the agent moves:

```
R1  ¬P11
R2  ¬W11
R3  B11 ⇔ (P12 ∨ P21)
R4  S11 ⇔ (W12 ∨ W21)
R5  B12 ⇔ (P11 ∨ P22)
R6  S12 ⇔ (W11 ∨ W22)
R7  B21 ⇔ (P11 ∨ P22)
R8  S21 ⇔ (W11 ∨ W22)
R9  B22 ⇔ (P12 ∨ P21)
R10 S22 ⇔ (W12 ∨ W21)
R11 W11 ∨ (W12 ∨ (W21 ∨ W22))
R12 ¬W11 ∨ ¬W12
R13 ¬W11 ∨ ¬W21
R14 ¬W11 ∨ ¬W22
R15 ¬W12 ∨ ¬W21
R16 ¬W12 ∨ ¬W22
R17 ¬W21 ∨ ¬W22
```

Percept scenarios designed for the queries

Scenario	Percept sequence (facts added to the KB)	Used for
A	[1,1]: no breeze, no stench → ¬B11, ¬S11	n = 2, 3, 4

Scenario	Percept sequence (facts added to the KB)	Used for
B	[1,1]: none; [2,1]: breeze, no stench (B21, ¬S21); [1,2]: none (¬B12, ¬S12)	n = 3, 4 (it is contradictory for n = 2, where B21 forces a pit on [2,2] while ¬B12 forbids it)

Part A – Student A: design and mathematical analysis

(i) Symbols and sentences as a function of n

Four symbols per square and n^2 squares give $\mathbf{m = 4n^2}$ symbols. The sentence count is the sum of the four groups: $\mathbf{N(n) = 2 + 2n^2 + 1 + n^2(n^2–1)/2 = (n^4 + 3n^2 + 6)/2}$. (Percept facts add at most $2n^2$ more sentences.) The program counts the generated sentences and they agree with the formula for every n shown:

n	symbols m = 4n²	models 2^m	sentences (formula)	sentences (generated)
2	16	65,536	17	17
3	36	6.87×10^10	57	57
4	64	1.84×10^19	155	155
5	100	1.27×10^30	353	353

(ii) Number of models and the symbols relevant to one query

A model assigns true/false to every symbol, so there are $\mathbf{2^m = 2^{(4n^2)}}$ models: 65,536 for n = 2 but 6.9×10^{10} for n = 3 and 1.8×10^{19} for n = 4. Only a handful of symbols can affect a given query, however. For the query "[2,2] is safe" (¬P22 ∧ ¬W22) in scenario B the **minimal relevant set is 8 symbols**: P22, P11, P13, B12 (the sentence B12 ⇔ P11 ∨ P13 ∨ P22 together with ¬B12 forces ¬P22) and W22, W11, W31, S21 (S21 ⇔ W11 ∨ W22 ∨ W31 together with ¬S21 forces ¬W22). That reduces 2^36 (or 2^64) models to 2^8 = 256. A looser but automatic choice is the **neighbourhood set**: the P/W symbols of every visited square and its neighbours plus the B/S symbols of visited squares (18 symbols in scenario B, 8 in scenario A). Restricting the KB to the sentences that mention only those symbols is sound for entailment, because KB′ ⊆ KB and entailment is monotonic: KB′ ⊨ α implies KB ⊨ α. It can be incomplete (a dropped sentence may be needed), which is why Student B compares the answers with the full set where that is feasible (n = 2).

(iii) Soundness and completeness of model checking

Soundness: the checker answers "KB ⊨ α" only if α is true in every model in which the KB is true – it checks exactly the definition of entailment, so it can never claim an entailment that does not hold. **Completeness:** the set of models is finite (2^m), so the enumeration terminates and finds every entailed α; whenever KB ⊨ α holds, the checker reports it. The price is time: Θ(2^m) models in the worst case, independent of how simple the query is.

Part A – Student B: experimental validation

The truth-table checker (`tt_entails`) compiles every sentence to a Python expression over a bit-mask, enumerates the 2^m models in order, evaluates the KB sentence by sentence with short-circuiting, and checks the query only in models where the KB is true. It records **models enumerated, KB evaluations (one per model), individual sentence evaluations, KB-true models and elapsed time**.

```
def tt_entails(kb, query, syms=None, instrument=True):
    """Truth-table entailment (TT-ENTAILS).  kb = list of sentences.
    Returns (entailed, stats).  stats: models enumerated, KB evaluations,
```

```
    sentence evaluations (short-circuit), KB-true models, query evaluations, seconds."""
    if syms is None:
        syms = set(symbols(query))
        for s in kb: symbols(s, syms)
    syms = sorted(syms)
    idx = {s: i for i, s in enumerate(syms)}
    fs = [compile_f(s, idx) for s in kb]
    q = compile_f(query, idx)
    models = kb_evals = sent_evals = kb_true = q_evals = 0
    t0 = time.perf_counter()
    entailed = True
    counter = None
    for b in range(1 << len(syms)):
        models += 1
        kb_evals += 1
        ok = True
        for f in fs:
            sent_evals += 1
            if not f(b):
                ok = False; break
        if ok:
            kb_true += 1
            q_evals += 1
            if not q(b):
                entailed = False; counter = b
                break
    dt = time.perf_counter() - t0
    st = dict(symbols=len(syms), total_models=1 << len(syms), models=models, kb_evals=kb_evals,
              sentence_evals=sent_evals, kb_true_models=kb_true, query_evals=q_evals, seconds=dt)
    if counter is not None:
        st['counter_model'] = {s: bool((counter >> idx[s]) & 1) for s in syms}
    return entailed, st
```

n	sc.	query (expected type)	FULL symbol set: m / models enumerated / time	NEIGHBOURHOOD set: m / models / time	MINIMAL set: m / models / time	verdict
2	A	[1,2] is safe (entailed)	16 / 65,536 / 19.2 ms	8 / 256 / 0.07 ms	6 / 64 / 0.05 ms	entailed
2	A	pit at [1,2] (negation entailed)	16 / 34,305 / 9.4 ms	8 / 1 / 0.00 ms	3 / 1 / 0.00 ms	negation entailed
2	A	pit at [2,2] (neither)	16 / 34,305 / 10.0 ms	9 / 1 / 0.00 ms	1 / 1 / 0.00 ms	neither
3	A	[1,2] is safe (entailed)	36 / 6.87×10^10 total / INFEASIBLE (est. 5.6 h)	8 / 256 / 0.07 ms	6 / 64 / 0.04 ms	entailed
3	A	pit at [1,2] (negation entailed)	36 / 6.87×10^10 total / INFEASIBLE (est. 5.6 h)	8 / 1 / 0.00 ms	3 / 1 / 0.00 ms	negation entailed
3	A	pit at [2,2] (neither)	36 / 6.87×10^10 total / INFEASIBLE (est. 5.6 h)	9 / 1 / 0.00 ms	1 / 1 / 0.00 ms	neither
3	B	[2,2] is safe (entailed)	36 / 6.87×10^10 total / INFEASIBLE (est. 5.6 h)	18 / 262,144 / 74.98 ms	8 / 256 / 0.08 ms	entailed
3	B	pit at [2,2] (negation entailed)	36 / 6.87×10^10 total / INFEASIBLE (est. 5.6 h)	18 / 261 / 0.13 ms	4 / 1 / 0.00 ms	negation entailed
3	B	pit at [3,2] (neither)	36 / 6.87×10^10 total / INFEASIBLE (est. 5.6 h)	19 / 261 / 0.14 ms	1 / 1 / 0.00 ms	neither
4	A	[1,2] is safe (entailed)	64 / 1.84×10^19 total / INFEASIBLE (est. 171,765 years)	8 / 256 / 0.07 ms	6 / 64 / 0.02 ms	entailed
4	A	pit at [1,2] (negation entailed)	64 / 1.84×10^19 total / INFEASIBLE (est. 171,765 years)	8 / 1 / 0.00 ms	3 / 1 / 0.00 ms	negation entailed
4	A	pit at [2,2] (neither)	64 / 1.84×10^19 total / INFEASIBLE (est. 171,765 years)	9 / 1 / 0.00 ms	1 / 1 / 0.00 ms	neither

n	sc.	query (expected type)	FULL symbol set: m / models enumerated / time	NEIGHBOURHOOD set: m / models / time	MINIMAL set: m / models / time	verdict
4	B	[2,2] is safe (entailed)	64 / 1.84×10^19 total / INFEASIBLE (est. 171,765 years)	18 / 262,144 / 100.05 ms	8 / 256 / 0.11 ms	entailed
4	B	pit at [2,2] (negation entailed)	64 / 1.84×10^19 total / INFEASIBLE (est. 171,765 years)	18 / 261 / 0.14 ms	4 / 1 / 0.01 ms	negation entailed
4	B	pit at [3,2] (neither)	64 / 1.84×10^19 total / INFEASIBLE (est. 171,765 years)	19 / 261 / 0.26 ms	1 / 1 / 0.00 ms	neither

Table 1.1 – Model-checking cost. "Models enumerated" counts models visited until the checker stopped: it stops at the first counter-model, so queries that are not entailed (and the "negation entailed" query) finish early, while an entailed query forces the whole space to be scanned. The verdict column was obtained by checking both α and ¬α.

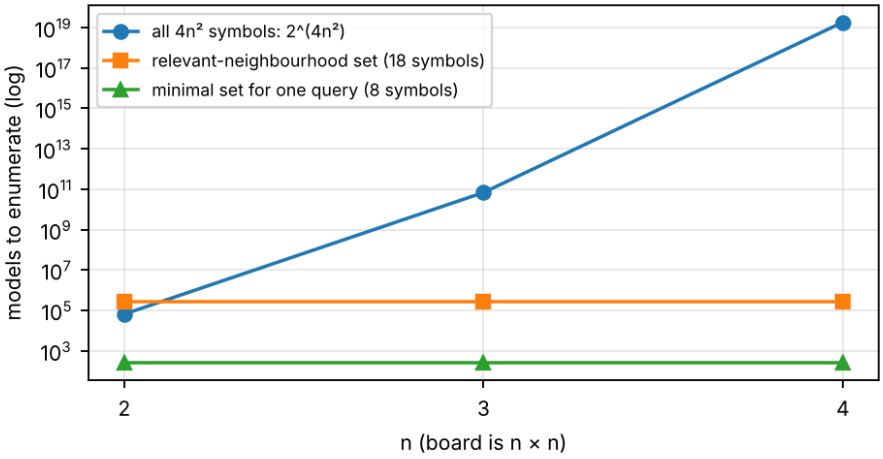


Figure 1.1 – Models that must be enumerated as the board grows.

Where the full symbol set becomes infeasible. For n = 2 the full set (16 symbols) is easy: 65,536 models and 131,334 sentence evaluations in 19 ms (≈ 3.4 million models/s). For n = 3 the full set has 36 symbols – 2^36 = 6.9×10[10] models, about **5.6 hours** at the measured rate – and for n = 4 it has 2^64 ≈ 1.8×10[19] models (≈ 171,765 years). The full-set experiment was therefore run only for n = 2 and is declared infeasible from n = 3 onward.

Effect of relevant-symbol restriction. With the neighbourhood set, the cost no longer depends on n at all: scenario B needs 262,144 models for n = 3 and exactly the same 262,144 for n = 4, because the same 18 symbols are relevant. With the hand-derived minimal set the entailed query needs only 256 models (544 sentence evaluations). For n = 2 the three symbol sets give the same verdicts as the full set, which is the evidence that the restriction was sound in these experiments.

The three query types. Entailed: "[1,2] is safe" (scenario A) and "[2,2] is safe" (scenario B). Negation entailed: "pit at [1,2]" (A) and "pit at [2,2]" (B) – the KB entails ¬α, so α is false in every model of the KB. Neither entailed: "pit at [2,2]" (A) and "pit at [3,2]" (B) – there are models of the KB in which the square holds a pit and models in which it does not.

Part B – Reasoning patterns: inference rules instead of model checking

Both students implemented derivations with the rules Modus Ponens (MP), And-elimination (AE) and Unit Resolution (UR). The rules are Python functions that **check their own applicability** (for example `mp` asserts

that the implication's antecedent equals the other premise) and append every application to a derivation log, so the number of inference steps is simply the length of the log. Two routine equivalence rewrites are also needed, and are counted as steps: Biconditional-elimination (a ⇔ b ≡ (a⇒b) ∧ (b⇒a)), Implication-elimination (a⇒b ≡ ¬a ∨ b) and De Morgan. A negative fact such as ¬P12 is obtained as: BE → AE → ImpE → UR (with the percept ¬B11) → De Morgan → AE.

Derivation 1 – squares [1,2] and [2,1] are safe (scenario A) – first 17 lines (square [1,2])

#	sentence	justification	from
1	¬P11	KB (start square safe)	–
2	B11 ⇔ (P12 ∨ P21)	KB rule R_B11	–
3	S11 ⇔ (W12 ∨ W21)	KB rule R_S11	–
4	¬B11	Percept at [1,1]	–
5	¬S11	Percept at [1,1]	–
6	(B11 ⇒ (P12 ∨ P21)) ∧ ((P12 ∨ P21) ⇒ B11)	Biconditional-elimination	2
7	(P12 ∨ P21) ⇒ B11	And-elimination	6
8	¬(P12 ∨ P21) ∨ B11	Implication-elimination (a⇒b ≡ ¬a∨b)	7
9	¬(P12 ∨ P21)	Unit Resolution	8,4
10	¬P12 ∧ ¬P21	De Morgan	9
11	¬P12	And-elimination	10
12	(S11 ⇒ (W12 ∨ W21)) ∧ ((W12 ∨ W21) ⇒ S11)	Biconditional-elimination	3
13	(W12 ∨ W21) ⇒ S11	And-elimination	12
14	¬(W12 ∨ W21) ∨ S11	Implication-elimination (a⇒b ≡ ¬a∨b)	13
15	¬(W12 ∨ W21)	Unit Resolution	14,5
16	¬W12 ∧ ¬W21	De Morgan	15
17	¬W12	And-elimination	16

Lines 6–17 derive ¬P12 (lines 6–11) and ¬W12 (lines 12–17): **12 inference steps** prove that [1,2] is safe. Repeating the pattern on the other disjunct gives ¬P21 and ¬W21, i.e. [2,1] safe, in **12 further steps** (no lemmas were shared, to keep the count honest).

Derivation 2 – square [2,2] is safe and a pit lies on [3,1] (scenario B)

#	sentence	justification	from
1	¬P11	KB (start square safe)	–
2	B12 ⇔ (P11 ∨ (P13 ∨ P22))	KB rule R_B12	–
3	S21 ⇔ (W11 ∨ (W22 ∨ W31))	KB rule R_S21	–
4	B21 ⇔ (P11 ∨ (P22 ∨ P31))	KB rule R_B21	–
5	¬B12	Percept at [1,2]	–
6	¬S21	Percept at [2,1]	–
7	B21	Percept at [2,1]	–
8	(B12 ⇒ (P11 ∨ (P13 ∨ P22))) ∧ ((P11 ∨ (P13 ∨ P22)) ⇒ B12)	Biconditional-elimination	2
9	(P11 ∨ (P13 ∨ P22)) ⇒ B12	And-elimination	8
10	¬(P11 ∨ (P13 ∨ P22)) ∨ B12	Implication-elimination (a⇒b ≡ ¬a∨b)	9
11	¬(P11 ∨ (P13 ∨ P22))	Unit Resolution	10,5
12	¬P11 ∧ ¬(P13 ∨ P22)	De Morgan	11

#	sentence	justification	from
13	¬(P13 ∨ P22)	And-elimination	12
14	¬P13 ∧ ¬P22	De Morgan	13
15	¬P22	And-elimination	14
16	(S21 ⇒ (W11 ∨ (W22 ∨ W31))) ∧ ((W11 ∨ (W22 ∨ W31)) ⇒ S21)	Biconditional-elimination	3
17	(W11 ∨ (W22 ∨ W31)) ⇒ S21	And-elimination	16
18	¬(W11 ∨ (W22 ∨ W31)) ∨ S21	Implication-elimination (a⇒b ≡ ¬a∨b)	17
19	¬(W11 ∨ (W22 ∨ W31))	Unit Resolution	18,6
20	¬W11 ∧ ¬(W22 ∨ W31)	De Morgan	19
21	¬(W22 ∨ W31)	And-elimination	20
22	¬W22 ∧ ¬W31	De Morgan	21
23	¬W22	And-elimination	22
24	(B21 ⇒ (P11 ∨ (P22 ∨ P31))) ∧ ((P11 ∨ (P22 ∨ P31)) ⇒ B21)	Biconditional-elimination	4
25	B21 ⇒ (P11 ∨ (P22 ∨ P31))	And-elimination	24
26	P11 ∨ (P22 ∨ P31)	Modus Ponens	25,7
27	P22 ∨ P31	Unit Resolution	26,1
28	P31	Unit Resolution	27,15

Lines 8–23 prove ¬P22 and ¬W22 (**16 steps**), so [2,2] is safe. Lines 24–28 use Modus Ponens (line 26, from B21 and B21 ⇒ P11∨P22∨P31) and two Unit Resolutions to conclude **P31** – a pit at [3,1] – in **5 steps**.

Student A – steps versus model-checking cost for the same queries

query	rule-based inference steps	model checking: full set	neighbourhood set	hand-derived minimal set
[1,2] is safe (A)	12	65,536 models / 131,334 sentence evals (n=2) – infeasible for n ≥ 3	256 models / 543 evals	64 models / 127 evals
[2,2] is safe (B)	16	infeasible (2^36, 2^64)	262,144 models / 532,790 evals	256 models / 544 evals
pit at [3,1] (B)	5	infeasible	not run (same 18-symbol set, 262,144 models)	not computed

Rule-based derivation is independent of n and needs 12–16 steps, against 10^2–10^5 model evaluations even when the symbol set has been carefully restricted. Its weakness is that a derivation has to be **found**: the program above replays derivations that the group designed by hand, whereas the truth-table checker needs no insight. A rule set of MP/AE/UR alone is also incomplete (it cannot, for example, derive entailed disjunctions that need case analysis), whereas model checking is complete.

Part B – Student B: soundness of the rules

A rule is **sound** if every model that satisfies all premises also satisfies the conclusion. The tables below enumerate all four assignments of (a, b); the last column shows that in every row where all premises are true the conclusion is true (the rows marked ✓). The program verified this for each rule (`sound = True`).

Modus Ponens: a ⇒ b, a ⊢ b

a	b	a ⇒ b	a	b	check
T	T	T	T	T	✓ premises true, conclusion true

a	b	a ⇒ b	a	b	check
T	F	F	T	F	(premises not all true)
F	T	T	F	T	(premises not all true)
F	F	T	F	F	(premises not all true)

And-elimination: a ∧ b ⊢ a

a	b	a ∧ b	a	check
T	T	T	T	✓ premises true, conclusion true
T	F	F	T	(premises not all true)
F	T	F	F	(premises not all true)
F	F	F	F	(premises not all true)

Unit Resolution: a ∨ b, ¬a ⊢ b

a	b	a ∨ b	¬a	b	check
T	T	T	F	T	(premises not all true)
T	F	T	F	F	(premises not all true)
F	T	T	T	T	✓ premises true, conclusion true
F	F	F	T	F	(premises not all true)

Biconditional-elimination: a ⇔ b ⊢ (a ⇒ b) ∧ (b ⇒ a)

a	b	a ⇔ b	(a ⇒ b) ∧ (b ⇒ a)	check
T	T	T	T	✓ premises true, conclusion true
T	F	F	F	(premises not all true)
F	T	F	F	(premises not all true)
F	F	T	T	✓ premises true, conclusion true

Implication-elimination: a ⇒ b ⊢ ¬a ∨ b

a	b	a ⇒ b	¬a ∨ b	check
T	T	T	T	✓ premises true, conclusion true
T	F	F	F	(premises not all true)
F	T	T	T	✓ premises true, conclusion true
F	F	T	T	✓ premises true, conclusion true

De Morgan: ¬(a ∨ b) ⊢ ¬a ∧ ¬b

a	b	¬(a ∨ b)	¬a ∧ ¬b	check
T	T	F	F	(premises not all true)
T	F	F	F	(premises not all true)
F	T	F	F	(premises not all true)
F	F	T	T	✓ premises true, conclusion true

Why a derived sentence holds in every model of the KB. Let M be any model in which every KB sentence is true. Induction on the length of the derivation: the initial lines (KB sentences and percepts) are true in M by assumption; each later line is the conclusion of a rule whose premises are earlier lines, hence true in M by the induction hypothesis, and the truth tables above show the conclusion must then be true in M. So every line – in particular the final one – is true in all models of the KB, i.e. KB $\models \alpha$. This is exactly the soundness of the derivation.

Why "not derived" is not "false". Failing to find a derivation means only that the particular, incomplete rule set did not reach the sentence. Even for a complete procedure, "KB $\not\models \alpha$" says that some model of the KB makes α false – it does not say that α is false in all models. Example (scenario A, query P22 "pit at [2,2]"): the relevant part of the KB has exactly **2 models**, and P22 is true in 1 of them and false in the other, so P22 can be neither derived nor refuted; the agent must treat [2,2] as *unknown*, not as safe and not as dangerous. The two models differ only in the value of P22:

model	B11	P11	P12	P21	P22	S11	W11	W12	W21
M1 (P22 true)	F	F	F	F	T	F	F	F	F
M2 (P22 false)	F	F	F	F	F	F	F	F	F

Comparison of the two reasoning approaches and cross-understanding

	Model checking	Inference rules (MP, AE, UR)
Cost	Θ(2^m) models; 256–262,144 even after restriction	12–16 steps, independent of n
Soundness / completeness	sound and complete (finite models)	sound; incomplete as a rule set
Needs insight?	No – fully automatic	Yes – derivation must be found or searched for
Result for unknown squares	proves both α and $\neg\alpha$ are not entailed, and exhibits models	silent: "not derived" carries no information

Cross-understanding. *Student A explains Student B's component:* the 2^m formula is why the full symbol set is infeasible at n = 3; the experiments confirm it (6.9×10^{10} models) and show that restriction removes the dependence on n. *Student B explains Student A's component:* the number of inference steps is small because each rule application is justified by a truth table, so the derivation is a short proof that every model of the KB makes the conclusion true; soundness of the rules (Part B) is what makes the cheap derivation trustworthy instead of a heuristic.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution (7 Marks)

Shared Group Work: Implement both a truth-table model checker and a propositional resolution prover (CNF conversion followed by resolution refutation). Design a smart-building access-control policy of at least 10 rules (badge validity, time of day, visitor status, emergency override) that contains implication, biconditional and disjunction.

The group must design test knowledge bases that make each method look strong or weak. Do not use a single sample KB.

Student A – Resolution Investigation

- Convert three sentences to CNF by hand and construct a resolution proof for one entailed query •

Record clauses generated, resolvents attempted, tautologies discarded, subsumed clauses removed and time as the KB grows (implication chains of length 10, 20, 40, 80)

- Test a query that is not entailed and explain why the process terminates

Student B – Model-Checking Comparative Analysis

- Record models enumerated and time for the same KBs and queries
- Identify a KB family on which model checking is faster than resolution and a family on which it is far slower, and explain what structural property causes the difference

Joint Design Question

If you were designing a reasoning component whose KB size and query types are not known in advance, what information would you want before deciding between these two methods? Support your answer with your own measurements.

Evidence Expected: Manual CNF conversion, proof tree, both implementations, clause and model counts across KB sizes, designed KB families and the joint design discussion.

ANSWER – TASK 2: Model Checking versus Resolution

Code: `prop.py` (truth-table checker `tt_entails`, compiled variant `tt_entails_fast`, CNF conversion `cnf_clauses`, resolution prover `resolution`) and `t2_compare.py` (policy, KB families, experiments). Raw outputs: `data/t2.json`.

Shared group work – implementations

- **Model checker.** Enumerates all 2^m truth assignments, evaluates the KB (short-circuiting) and checks the query in the KB-true models; counts models enumerated and KB-true models and records time.
- **Resolution prover.** CNF conversion (eliminate ⇔ and ⇒, push ¬ inwards, distribute ∨ over ∧, drop tautologies and duplicates), then refutation: the CNF of KB ∧ ¬query is saturated by given-clause resolution with a literal index. Each new clause is checked for **tautology** (discarded), **duplicates** and **forward/backward subsumption** (removed). It counts clauses generated, resolvents attempted (every resolution operation), tautologies discarded, subsumed clauses removed and time, and keeps parent links to extract the proof.

```
def resolution(kb, query, max_clauses=400000, strategy='bfs', time_limit=None):
    """Propositional resolution refutation: proves KB |= query by deriving {} from CNF(KB & ~query).
    Counters: generated (distinct kept resolvents), attempted (resolution operations),
    tautologies discarded, subsumed removed (forward + backward), proof extraction by parent links."""
    import heapq
    clauses = []
    for s in kb: clauses += cnf_clauses(s)
    clauses += cnf_clauses(('not', query))
    stats = dict(generated=0, attempted=0, tautologies=0, subsumed=0, initial=0, seconds=0.0)
    t0 = time.perf_counter()
    cid = {}; parents = {}; cl = []     # id -> clause
    processed = []; idx = {}            # literal -> set(ids)
    alive = set()
    agenda = []; seen = set()
    cnt = itertools.count()
    def push(c, par):
        if c in seen: return None
        seen.add(c)
        i = len(cl); cl.append(c); parents[i] = par
```

```
        key = (len(c), next(cnt)) if strategy == 'unit' else (next(cnt),)
        heapq.heappush(agenda, (key, i)); return i
    for c in clauses: push(c, None)
    stats['initial'] = len(cl)
    found = None
    while agenda:
        if time_limit and time.perf_counter() - t0 > time_limit: stats['status'] = 'timeout'; break
        _, i = heapq.heappop(agenda); c = cl[i]
        if not c: found = i; break
        # forward subsumption
        sub = False
        cand = set.intersection(*[idx.get(l, set()) for l in c]) if False else None
        for j in list(alive):
            if cl[j] <= c: sub = True; break
        if sub: stats['subsumed'] += 1; continue
        # backward subsumption
        for j in list(alive):
            if c < cl[j]:
                alive.discard(j); stats['subsumed'] += 1
                for l in cl[j]: idx[l].discard(j)
        alive.add(i)
        for l in c: idx.setdefault(l, set()).add(i)
        for l in c:
            for j in list(idx.get(neg(l), ())):
                if j not in alive: continue
                d = cl[j]
                stats['attempted'] += 1
                r = (c - {l}) | (d - {neg(l)})
                if is_taut(r): stats['tautologies'] += 1; continue
                k = push(r, (i, j))
                if k is not None:
                    stats['generated'] += 1
                    if not r: found = k; break
            if found is not None: break
        if found is not None: break
        if len(cl) > max_clauses: stats['status'] = 'cap'; break
```

Smart-building access-control policy (12 rules)

Symbols (13): Bv valid badge, Emp employee, Vis visitor, Esc escorted, Wh working hours, Ah after hours, Em emergency, Ov override active, Au area-authorised, Gr grant access, Dn deny access, Al alarm, Lg log entry.

rule	sentence	meaning
R1	Ah ⇔ ¬Wh	After hours is the same as not working hours
R2	Emp ⇔ ¬Vis	A person is an employee exactly when not a visitor
R3	((Bv ∧ Emp) ∧ Wh) ⇒ Gr	Valid badge + employee + working hours => grant
R4	(((Bv ∧ Emp) ∧ Ah) ∧ Au) ⇒ Gr	Valid badge + employee + after hours + authorised => grant
R5	(((Bv ∧ Vis) ∧ Esc) ∧ Wh) ⇒ Gr	Valid badge + escorted visitor + working hours => grant
R6	Ov ⇔ Em	Override is active exactly when there is an emergency
R7	Ov ⇒ Gr	Emergency override => grant
R8	Dn ⇔ ¬Gr	Deny exactly when not granted
R9	(Vis ∧ ¬Esc) ⇒ (Dn ∨ Ov)	Unescorted visitor => deny or override
R10	Dn ⇒ Lg	Every denial is logged
R11	Al ⇔ (Em ∨ (Dn ∧ Ah))	Alarm iff emergency or (denial after hours)
R12	(¬Bv ∧ ¬Ov) ⇒ Dn	No valid badge and no override => deny

The policy contains **implications** (R3–R5, R7, R9, R10, R12), **biconditionals** (R1, R2, R6, R8, R11) and **disjunctions** (R9 and R11).

Student A – resolution investigation

(a) Hand conversion of three sentences to CNF

R8: Dn ⇔ ¬Gr. Eliminate ⇔: (Dn ⇒ ¬Gr) ∧ (¬Gr ⇒ Dn). Eliminate ⇒: (¬Dn ∨ ¬Gr) ∧ (¬¬Gr ∨ Dn) = (¬Dn ∨ ¬Gr) ∧ (Gr ∨ Dn). Clauses: {¬Dn, ¬Gr}, {Dn, Gr}.

R9: (Vis ∧ ¬Esc) ⇒ (Dn ∨ Ov). Eliminate ⇒: ¬(Vis ∧ ¬Esc) ∨ Dn ∨ Ov. De Morgan: ¬Vis ∨ ¬¬Esc ∨ Dn ∨ Ov = ¬Vis ∨ Esc ∨ Dn ∨ Ov. One clause: {¬Vis, Esc, Dn, Ov}.

R11: Al ⇔ (Em ∨ (Dn ∧ Ah)). Eliminate ⇔: [Al ⇒ (Em ∨ (Dn ∧ Ah))] ∧ [(Em ∨ (Dn ∧ Ah)) ⇒ Al]. First half: ¬Al ∨ Em ∨ (Dn ∧ Ah); distribute ∨ over ∧: (¬Al ∨ Em ∨ Dn) ∧ (¬Al ∨ Em ∨ Ah). Second half: ¬(Em ∨ (Dn ∧ Ah)) ∨ Al = (¬Em ∧ (¬Dn ∨ ¬Ah)) ∨ Al; distribute: (¬Em ∨ Al) ∧ (¬Dn ∨ ¬Ah ∨ Al). Four clauses: {¬Al, Em, Dn}, {¬Al, Em, Ah}, {¬Em, Al}, {¬Dn, ¬Ah, Al}.

The program's CNF converter produced exactly these clauses: R8 → {~Dn, ~Gr}; {Dn, Gr}. R9 → {Dn, Esc, Ov, ~Vis}. R11 → {~Al, Dn, Em}; {Ah, ~Al, Em}; {Al, ~Em}; {~Ah, Al, ~Dn}.

(b) Resolution proof of one entailed query

Query Q1: with the facts Bv, Emp, Wh, ¬Em added to the policy, is **Gr** (access granted) entailed? Negating the query adds {¬Gr}. The prover derived the empty clause; the extracted proof (only the clauses that take part, ids as in the program) is:

id	clause	origin
4	{~Bv, ~Emp, Gr, ~Wh}	input
19	{Bv}	input
20	{Emp}	input
21	{Wh}	input
23	{~Gr}	input
60	{~Emp, Gr, ~Wh}	resolve 19, 4
183	{~Emp, Gr}	resolve 60, 21
215	{Gr}	resolve 183, 20
222	{} (empty clause)	resolve 215, 23

Proof tree (read top-down; each node is the resolvent of its two children):

```
{}  (empty clause)      [resolve 215 , 23]
  └─ {Gr}      [resolve 183 , 20]
     └─ {~Emp, Gr}      [resolve 60 , 21]
        └─ {~Emp, Gr, ~Wh}      [resolve 19 , 4]
           └─ {Bv}
           └─ {~Bv, ~Emp, Gr, ~Wh}
        └─ {Wh}
     └─ {Emp}
  └─ {~Gr}
```

The search generated 199 distinct clauses and attempted 322 resolutions before finding the 4-step proof (the proof uses only a small fraction of the search effort – typical for saturation without a goal-directed strategy).

(c) The whole policy: both methods on seven queries

q	query	MC	models	ms	Res	generated	attempted	taut.	subsumed	ms
Q1	employee, valid badge, working hours => grant	yes	8,192	2.09	yes	199	322	44	194	1.5
Q2	invalid badge, no emergency => deny	yes	8,192	2.07	yes	184	307	50	103	1.1
Q3	unescorted visitor => deny	yes	8,192	2.08	yes	188	316	32	168	2.6
Q4	emergency => grant	yes	8,192	2.05	yes	172	288	64	41	1.0
Q5	after hours employee => grant (NOT entailed: Au unknown)	no	604	0.19	no	212	342	40	224	1.5
Q6	escorted visitor => log entry (NOT entailed)	no	6,537	1.61	no	159	290	54	168	1.2
Q7	denial after hours => alarm	yes	8,192	2.00	yes	174	268	44	145	1.1

*Table 2.1 – The 12-rule policy (13 symbols, 8,192 models). Both methods agree on all seven queries; Q5 and Q6 are deliberately **not** entailed (the policy never mentions the authorisation Au of an after-hours employee, nor a log entry for granted visitors).*

(d) Growth with KB size: implication chains of length 10, 20, 40, 80

chain n	symbols	clauses generated	resolvents attempted	tautologies discarded	subsumed removed	time (ms)	proof steps	attempted (shortest-clause-first)	time (ms)
10	11	66	153	0	34	0.47	11	12	0.10
20	21	231	908	0	119	3.34	21	22	0.25
40	41	861	6180	0	445	25.95	41	42	0.56
80	81	3321	45374	0	1721	255.05	81	82	1.14

Table 2.2 – Resolution on KB = {A0, A0⇒A1, …, A(n−1)⇒An}, query An. The right-most two columns repeat the experiment with a unit/shortest-clause-first agenda.

Resolvents attempted grow roughly quadratically (153 → 908 → 6,180 → 45,374: a factor of 6–7 per doubling of n, i.e. growth like n^2.8), because resolving the chain clauses against each other produces all "shortcut" clauses ¬Ai ∨ Aj before they are subsumed by the derived units; yet the proof itself is always n+1 steps long and the total time stays under 0.3 s for n = 80. Processing the **shortest clause first** (unit preference) removes the blow-up completely: only 12, 22, 42 and 82 resolvents are attempted (n + 2), because each new unit Ai is resolved at once and subsumes the shortcut clauses before they are generated, and n = 80 takes about 1 ms. The agenda order, not the inference rule, is what makes the baseline expensive.

(e) A query that is not entailed – why the process terminates

chain n	entailed?	prover status	clauses generated	resolvents attempted	time (ms)
10	no	saturated	55	172	0.62
20	no	saturated	210	1129	4.07
40	no	saturated	820	8095	35.82
80	no	saturated	3240	61115	397.96

Query “B_unrelated” (a symbol not in the KB). **Why it terminates:** over m symbols there are at most 3^m distinct clauses (each symbol is absent, positive or negative in a clause). The prover discards duplicates and tautologies, so every resolvent it keeps is a clause it has never held before – the set of clauses can only grow by finitely many steps and then no new resolvent exists (“saturation”). Since the empty clause was not derived and resolution is sound, KB ∧ ¬query is satisfiable and therefore KB ⊭ query. Completeness of resolution **refutation** guarantees the converse: if KB ⊨ query, saturation would have produced the empty clause. The same happened on the policy queries Q5 and Q6 (status “saturated”).

Student B – model-checking comparative analysis

The same KBs and queries were given to the truth-table checker (compiled form `tt_entails_fast`, one bit-mask per model). Results for the chains (rows from Table 2.2):

chain n	models 2^(n+1)	models enumerated	model checking (ms)	resolution (ms)	MC slower by
10	2.05e+03	2,048	0.3	0.47	1×
20	2.1e+06	2,097,152	336.8	3.34	101×
40	2.2e+12	infeasible	–	25.95	$>10^9$×
80	2.42e+24	infeasible	–	255.05	$>10^9$×

Family 1 (model checking far slower): sparse Horn implication chains

The truth table doubles with every new symbol: at n = 20 it needs 2,097,152 models (0.34 s against 3 ms), at n = 40 it needs 2.2×10^{12} models (not run, ≈ 4 days at the 6 million models/s measured here) and at n = 80 2.4×10^{24} models. Resolution works on the **structure**: every clause has two literals, one resolution step advances the chain by one link, and the proof is n+1 steps long.

Family 2 (model checking faster): dense KBs on few symbols

KB (unsatisfiable, so every query is entailed)	symbols	sentences	models 2^m	model checking (ms)	resolution (ms)	clauses generated
pigeonhole n=2 (3 pigeons, 2 holes)	7	9	128	0.0	1.1	64
pigeonhole n=3 (4 pigeons, 3 holes)	13	22	8,192	3.2	3116.8	59,699
pigeonhole n=4 (5 pigeons, 4 holes)	21	45	2,097,152	1206.9	5424.5 (aborted at 400k clauses)	400,187
pigeonhole n=5 (6 pigeons, 5 holes)	31	81	2,147,483,648	not run	4546.4 (aborted at 400k clauses)	400,064
random 3-CNF, 8 variables, 6 clauses/var	9	48	512	0.03	48	2,616
random 3-CNF, 10 variables, 6 clauses/var	11	60	2,048	1.10	782	20,447
random 3-CNF, 12 variables, 6 clauses/var	13	72	8,192	4.04	3708	106,119
random 3-CNF, 14 variables, 6 clauses/var	15	84	32,768	22.90	8923 (aborted at 400k clauses)	400,064
random 3-CNF, 16 variables, 6 clauses/var	17	96	131,072	71.55	4573 (aborted at 400k clauses)	399,973

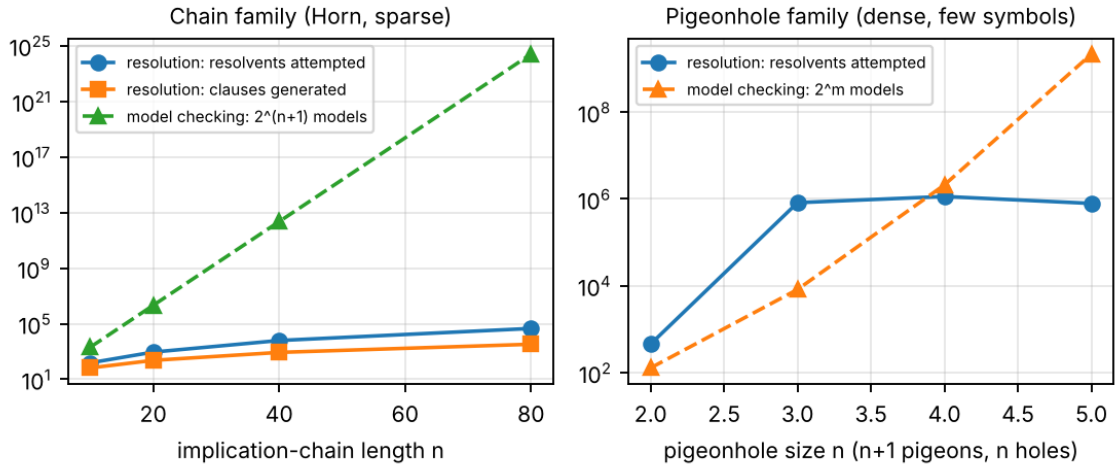


Figure 2.1 – Left: on Horn chains resolution grows polynomially while the model count is exponential. Right: on the dense pigeonhole family the resolution effort grows faster than the (already exponential) model count at small sizes.

Structural property. The cost of model checking depends only on the **number of symbols m** (2^m), however many sentences there are – 8,192 models for the 13-symbol KBs above, whether the KB has 22 or 96 clauses. The cost of resolution depends on the **clause structure**. A sparse Horn KB (short clauses, each symbol in few clauses) has linear-size proofs and polynomially many useful resolvents. A dense non-Horn KB over few symbols (pigeonhole, random 3-CNF) has many overlapping clauses; resolution then derives vast numbers of resolvents and tautologies (220,320 tautologies for pigeonhole-3), and for the pigeonhole family it is known that every resolution proof has exponential size (Haken, 1985). On 12 random variables model checking took 4 ms and resolution 3.7 s – about 900 times longer – and on random 3-CNF with 14 and 16 variables resolution exhausted a 400,000-clause budget while model checking finished in 23 and 72 ms. On the other hand, when m grows too (pigeonhole-5: 31 symbols, 2×10^9 models) both methods fail in our implementation, which shows that neither dominates.

Joint design question – what to know before choosing a method

My measurements suggest asking for the following information, in this order:

1. **Number of distinct symbols m** (cheapest, decisive). Model checking costs 2^m: 0.34 s at m = 21, ≈ 3 s at m = 24, hopeless at m ≥ 35. If m ≤ ~20 the cost is bounded and known in advance whatever the KB looks like.
2. **Structure of the sentences:** Horn/definite clauses and sparse implication networks (chains, rule bases) versus dense disjunctive constraints. For Horn KBs resolution – or better forward/backward chaining, which is linear – scales to hundreds of symbols (n = 80: 81 symbols in 0.25 s). For dense non-Horn constraints over few symbols model checking wins by 2–3 orders of magnitude (random 3-CNF, m = 12: 4 ms against 3.7 s).
3. **Query load and update pattern:** how many queries per KB version. Model checking can reuse one pass over the 2^m models for all queries (enumerate once, test many), resolution restarts for each query.
4. **Kind of query:** mostly entailed (proof wanted) or mostly not entailed? A non-entailed query makes resolution saturate the clause space (Q5/Q6 explored 159–212 clauses on 13 symbols), whereas

model checking stops at the first counter-model (604 models for Q5 against 8,192 for an entailed query) and returns an explicit counter-example.

5. **Need for explanations and for incremental growth:** resolution produces a checkable proof tree; model checking produces counter-models but no proof for entailed queries.

Decision rule from the evidence: estimate m and the fraction of Horn clauses when the KB is loaded; use model checking when m ≤ 20–25 and the KB is dense, use resolution (or chaining for Horn KBs) otherwise, and run the cheaper method under a time/clause budget with the other as fall-back – the pigeonhole and random experiments show that a budget is necessary because resolution can blow up silently.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic (7 Marks)

Problem Context – University Exam-Eligibility Advisor

Regulations decide whether a student may write an examination (minimum attendance, prerequisites cleared, fees paid, disciplinary status, approved leave).

Shared Group Work: Follow the knowledge-engineering process (identify the task, assemble knowledge, decide vocabulary, encode general knowledge, encode the instance, pose queries, debug the KB). Build two encodings of the same regulations: a **propositional** one (one set of rules per student and course) and a **first-order** one (general axioms plus facts).

Student A – Design Derivation

- Define the FOL vocabulary and write at least 12 axioms with quantifiers, including one existential statement
- Derive how the number of sentences in the propositional encoding grows with students *s* and courses *c*, and show how the FOL axioms remain constant

Student B – Crossover Investigation

- Design at least 15 competency queries (5 that succeed, 5 that fail, 5 returning bindings) and document at least three KB defects found and fixed
- Measure KB size and inference time for both encodings as *s* and *c* grow, and locate the range of sizes at which the FOL encoding becomes preferable despite unification overhead

Joint Conclusion

Why does greater expressiveness not automatically give better reasoning performance for every KB size? Support your answer with your measurements.

Evidence Expected: Vocabulary and axioms, growth derivation, query suite with expected and actual results, defect log, size/time tables and a reasoned conclusion.

ANSWER – TASK 3: Knowledge Engineering in First-Order Logic (University Exam-Eligibility Advisor)

Code: `fol.py` (terms, unification, KB with indexes, backward chaining with negation-as-failure) and `t3_exam.py` (both encodings, query suite, defect iterations, scaling experiments). Raw results: `data/t3.json`.

Shared group work – the knowledge-engineering process

Step	What the group did
1. Identify the task	Decide, for a student x and a course c, whether x may write the exam (Eligible), why not (Detained, MissingPrereq, Barred, FeeDefaulter) and issue a hall ticket; list the exam venue of a course.
2. Assemble knowledge	Regulations: attendance ≥ 75 %; attendance 65–74 % condoned with approved leave; all prerequisite courses passed (grade ≥ 40); fees paid; no unresolved disciplinary action (an upheld appeal lifts the bar).
3. Decide vocabulary	Predicates, constants and one function symbol (below). Numeric tests use the built-ins ge, gt, lt.
4. Encode general knowledge	Fourteen axioms A1–A14 (first-order) or, in the propositional encoding, one set of rules per student and per course.
5. Encode the instance	Facts: Enrolled, Attendance, Grade, ApprovedLeave, FeesPaid, Disciplined, AppealUpheld, Prereq, OffersExam. The small hand-made instance has 8 students and 4 courses; the generator creates s students and c courses (seeded random).
6. Pose queries	15 competency queries with hand-derived expected answers (below).
7. Debug the KB	Run the suite on successive drafts; three defects found and fixed (defect log).

Modelling assumption. The student records are treated as a complete database (closed world): a fact that is absent is false, and ¬P in a rule body is implemented as negation as failure. This is what makes "not passed", "not disciplined" or "no approved leave" computable; the rules are stratified (Barred, Passed < MissingPrereq < Eligible < Detained), so the negation is well defined.

Student A – design derivation

(i) First-order vocabulary

Symbol	Arity	Meaning
Student constants S1…Sn, course constants C1…Cc	–	individuals
Enrolled(x,c)	2	student x is enrolled in course c
Attendance(x,c,v)	3	x attended v percent of course c
Grade(x,c,g)	3	x obtained grade g in c (earlier attempt)
Prereq(p,c)	2	p is a prerequisite of c
ApprovedLeave(x,c)	2	x has approved leave in c
FeesPaid(x), Disciplined(x), AppealUpheld(x)	1	record facts about student x
OffersExam(c)	1	an exam is held for c
AttOK, Condoned, Passed, MissingPrereq, PrereqOK, Barred	2 / 1	derived predicates used inside the rules
Eligible(x,c), Detained(x,c), HallTicket(x,c), FeeDefaulter(x)	2 / 1	answers the advisor produces
Venue(c,r) and the Skolem function venue(c)	2 / 1	room r where the exam for c is held; venue/1 names "the" room
ge, gt, lt	2	built-in numeric comparisons

(ii) Axioms (14, with quantifiers; one existential statement)

#	axiom
A1	∀x,c Enrolled(x,c) ∧ AttOK(x,c) ∧ PrereqOK(x,c) ∧ FeesPaid(x) ∧ ¬Barred(x) ⇒ Eligible(x,c)
A2	∀x,c,v Attendance(x,c,v) ∧ v ≥ 75 ⇒ AttOK(x,c)
A3	∀x,c Condoned(x,c) ⇒ AttOK(x,c)
A4	∀x,c,v Attendance(x,c,v) ∧ 65 ≤ v < 75 ∧ ApprovedLeave(x,c) ⇒ Condoned(x,c)
A5	∀x,c,g Grade(x,c,g) ∧ g ≥ 40 ⇒ Passed(x,c)
A6	∀x,c [Enrolled(x,c) ∧ ∃p (Prereq(p,c) ∧ ¬Passed(x,p))] ⇒ MissingPrereq(x,c) (existential statement)
A7	∀x,c Enrolled(x,c) ∧ ¬MissingPrereq(x,c) ⇒ PrereqOK(x,c)
A8	∀x Disciplined(x) ∧ ¬AppealUpheld(x) ⇒ Barred(x)
A9	∀x,c Enrolled(x,c) ∧ ¬Eligible(x,c) ⇒ Detained(x,c)
A10	∀x,c Eligible(x,c) ⇒ HallTicket(x,c)
A11	∀x,c Enrolled(x,c) ∧ ¬FeesPaid(x) ⇒ FeeDefaulter(x)
A12	∀c OffersExam(c) ⇒ ∃r Venue(c,r) (Skolemised: Venue(c, venue(c)))
A13	∀x,c Eligible(x,c) ⇒ Enrolled(x,c) (integrity constraint, checked by test, not executed)
A14	∀x,c ¬(Eligible(x,c) ∧ Detained(x,c)) (integrity constraint, checked by test)

A6 and A12 contain existential quantifiers. A6 says that x is missing a prerequisite if **there exists** a prerequisite p of c that x has not passed; the existential is eliminated into a body variable because it sits in the antecedent. A12 is Skolemised to the function venue(c). A13 and A14 are integrity constraints that are checked by tests rather than executed.

(iii) Growth of the propositional encoding

The propositional encoding has to repeat each general rule for every constant it could apply to. Per student–course pair there are **9 rules**: Eligible, AttOK (≥ 75 %), AttOK (via condonation), Condoned, Passed, PrereqOK, Detained, HallTicket and FeeDefaulter. In addition there is one MissingPrereq rule **per student per prerequisite edge** (s·E, where E is the number of Prereq facts), one Barred rule per student (s) and one Venue rule per course (c). Hence

R_prop(s, c) = 9·s·c + s·E + s + c (with E ≈ 2c in our generator: R_prop = Θ(s·c)),

whereas the first-order encoding has **12 executable rules (14 axioms) for every s and c**: the number of general sentences is constant, and only the *facts* (the instance data) grow – and they grow in both encodings. The formula was checked against the generated propositional KB:

s	c	E	s·c	propositional rules (generated)	formula 9sc+sE+s+c	FOL rules	FOL axioms
5	3	2	15	153	153	12	14
10	5	6	50	525	525	12	14
25	5	6	125	1,305	1,305	12	14
50	10	16	500	5,360	5,360	12	14
100	10	16	1000	10,710	10,710	12	14
200	10	16	2000	21,410	21,410	12	14
500	10	16	5000	53,510	53,510	12	14
1000	10	16	10000	107,010	107,010	12	14
500	20	36	10000	108,520	108,520	12	14

s	c	E	s·c	propositional rules (generated)	formula 9sc+sE+s+c	FOL rules	FOL axioms
1000	20	36	20000	217,020	217,020	12	14

Both encodings were also verified to give **identical answers** for Eligible, Detained and HallTicket on every student–course pair: 96 / 96 pairs agree on the hand-made instance and 720 / 720 on a generated instance (40 students × 6 courses × 3 predicates). The integrity constraints A13 and A14 had 0 violations on a generated instance of 60 × 8.

Student B – crossover investigation

(i) The 15 competency queries – expected and actual results (final KB)

id	kind	query	expected	actual	result
Q1	succeed	Eligible(S1, C3)	true	true	PASS
Q2	succeed	Eligible(S2, C1)	true	true	PASS
Q3	succeed	Eligible(S5, C2)	true	true	PASS
Q4	succeed	Eligible(S6, C1)	true	true	PASS
Q5	succeed	Eligible(S8, C4)	true	true	PASS
Q6	fail	Eligible(S2, C3)	false	false	PASS
Q7	fail	Eligible(S3, C1)	false	false	PASS
Q8	fail	Eligible(S4, C1)	false	false	PASS
Q9	fail	Eligible(S7, C1)	false	false	PASS
Q10	fail	HallTicket(S4, C1)	false	false	PASS
Q11	bindings	Eligible(?x, C1)	Eligible(S1, C1), Eligible(S2, C1), Eligible(S6, C1)	Eligible(S1, C1), Eligible(S2, C1), Eligible(S6, C1)	PASS
Q12	bindings	Detained(?x, C1)	Detained(S3, C1), Detained(S4, C1), Detained(S7, C1)	Detained(S3, C1), Detained(S4, C1), Detained(S7, C1)	PASS
Q13	bindings	MissingPrereq(S2, ?c)	MissingPrereq(S2, C3)	MissingPrereq(S2, C3)	PASS
Q14	bindings	HallTicket(S1, ?c)	HallTicket(S1, C1), HallTicket(S1, C2), HallTicket(S1, C3)	HallTicket(S1, C1), HallTicket(S1, C2), HallTicket(S1, C3)	PASS
Q15	bindings	Venue(C2, ?r)	Venue(C2, venue(C2))	Venue(C2, venue(C2))	PASS

Table 3.1 – Query suite on the 8-student, 4-course instance. Queries Q1–Q5 succeed, Q6–Q10 fail, Q11–Q15 return bindings (Q15 returns the Skolem term venue(C2)).

(ii) Defect log – three defects found and fixed

draft	change	queries passed	failing queries
v0	first draft containing D1, D2, D3	7 / 15	Q2, Q3, Q4, Q6, Q11, Q12, Q13, Q14
v1	D2 fixed	10 / 15	Q2, Q8, Q10, Q11, Q12
v2	D2 and D1 fixed	12 / 15	Q2, Q11, Q12
v3	D2, D1 and D3 fixed (final)	15 / 15	–

defect	symptom found by the queries	cause	fix
D2 – prerequisites	Q2, Q3, Q4, Q11, Q13 fail: students of courses with no prerequisite (C1, C2) are never eligible; Q6 and Q13 show wrong MissingPrereq results; Q14 lacks HallTicket(S1,C3)	PrereqOK was written as "some prerequisite has been passed", an ∃ where a ∀ is required. It is false for a course without prerequisites (vacuous truth is lost) and true when only one of several prerequisites has been passed	A6/A7: MissingPrereq(x,c) holds if ∃p (Prereq(p,c) ∧ ¬Passed(x,p)); PrereqOK(x,c) ⇔ ¬MissingPrereq(x,c)
D1 – disciplinary bar	After D2 was fixed, Q8 (Eligible(S4,C1) should be false) and Q10 (HallTicket(S4,C1) should be false) failed	A1 omitted ¬Barred(x). It had been invisible in v0 because D2 made every course without prerequisites ineligible, so Q8 "passed for the wrong reason"	added ¬Barred(x) to A1
D3 – attendance boundary	Q2 (S2 has exactly 75 %) fails; Q11 and Q12 give wrong binding sets	A2 used v > 75 instead of v ≥ 75	A2 uses v ≥ 75; boundary cases (75, 65, 74) were added to the tests

The log shows two methodological points. First, defects mask each other: the pass count rose only 7 → 10 → 12 → 15, and D1 could be seen only after D2 was repaired. Second, tests need both "should succeed" and "should fail" queries – a suite with only positive queries would have missed D1 completely.

(iii) Size and inference time of both encodings as s and c grow

s	c	s·c	prop. rules	FOL rules	prop. total sentences	FOL total sentences	build prop (ms)	build FOL (ms)	query prop (µs)	query FOL (µs)	prop. steps/query	FOL unifications/query
5	3	15	153	12	193	64	0.11	0.09	1.8	110	7.9	15
10	5	50	525	12	648	166	0.33	0.16	1.8	107	7.4	18
25	5	125	1,305	12	1,589	373	0.72	0.32	2.2	115	7.8	19
50	10	500	5,360	12	6,344	1,292	3.51	1.00	2.5	126	6.9	29
100	10	1,000	10,710	12	12,709	2,551	7.35	2.11	2.7	128	7.3	29
200	10	2,000	21,410	12	25,402	5,061	14.73	3.65	3.5	142	7.6	34
500	10	5,000	53,510	12	63,457	12,538	47.84	8.90	4.1	151	8.4	34
1000	10	10,000	107,010	12	126,793	24,916	120.60	18.71	5.0	137	7.5	31
500	20	10,000	108,520	12	127,975	24,543	119.62	21.72	5.1	177	7.2	56
1000	20	20,000	217,020	12	255,680	48,811	361.49	36.82	6.3	191	7.9	58

Table 3.2 – Mean over 120–200 random Eligible(s,c) queries per size (backward chaining with indexing in both encodings; answers identical in every run). "Build" = constructing the KB from the data: grounding the regulations for the propositional KB, loading facts plus 12 rules for the FOL KB.

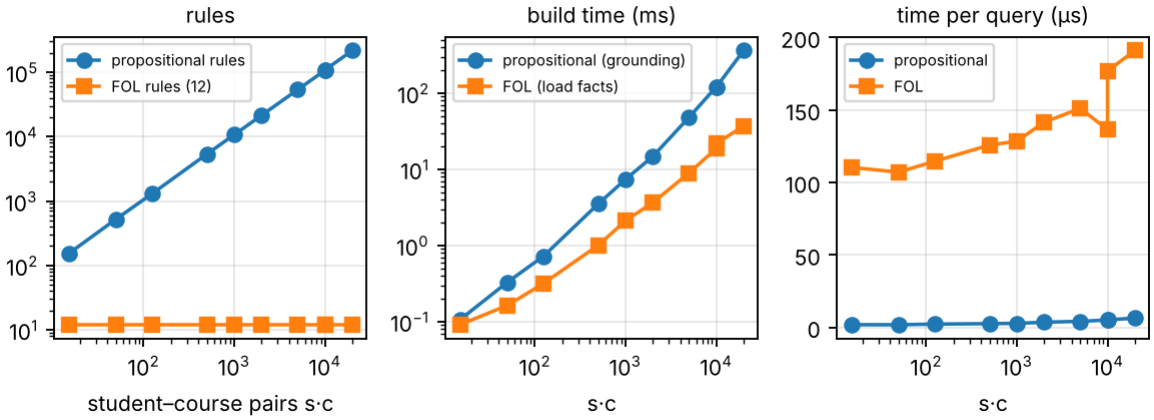


Figure 3.1 – Rules, build time and per-query time of the two encodings.

s×c	FOL KB memory (MB)	propositional KB memory (MB)	ratio
10x5	0.03	0.14	4×
100x10	0.28	3.06	11×
500x10	1.33	16.30	12×
1000x20	4.44	67.12	15×

Locating the crossover. Three different measures give three different answers:

- **Rule count:** the FOL encoding is smaller from the very first size (153 propositional rules against 12 already at 5 students × 3 courses) and the gap grows linearly (217,020 against 12 at 1000 × 20); counting the facts too, the FOL KB is 5.2× smaller at 1000 × 20 (48,811 against 255,680 sentences) because the facts are common to both.
- **Memory and build/update time:** FOL memory is 4.3× smaller at 10 × 5 and 15× smaller at 1000 × 20. Building the KB takes about the same time at the smallest size (0.09 vs 0.11 ms) and is 2.6–8.9× faster for the FOL encoding from s·c ≈ 500 onward (5–9× in most rows; 0.036 vs 0.32 s at 1000 × 20).
- **Inference time per query – no crossover:** the propositional encoding is 27–62× faster at *every* size (1.7–6.7 µs against 111–223 µs), because a ground rule is found by a dictionary lookup (≈ 8 steps per query) while the FOL engine pays for unification (15–58 unifications per query, ≈ 2.5 µs each against 0.03 µs for a dictionary lookup). The per-query curves are flat in s·c for both encodings, since both index their facts.

The practical break-even is therefore a **number of queries per KB version**: the propositional total cost is build + k·t_prop, the FOL cost is build + k·t_FOL, so FOL is cheaper as long as k < k* = (build_prop − build_FOL)/(t_FOL − t_prop). Measured values:

s	c	k* (queries per KB version)
5	3	0
10	5	2
25	5	4
50	10	20
100	10	42
200	10	80

s	c	k* (queries per KB version)
500	10	265
1000	10	775
500	20	570
1000	20	1,758

For example, at 1000 students × 20 courses the FOL encoding wins in total cost until about 1,760 queries have been answered per KB version; at the sizes 5×3 to 25×5, where grounding costs almost nothing, k* is below 4 (the propositional encoding is already better after a handful of queries). In a university the regulations and enrolments change every semester, and the KB is rebuilt each time, so FOL is the preferable encoding for large sizes ($s \cdot c \gtrsim 10^4$) unless an enormous number of queries is expected per rebuild; for a small, static KB the propositional encoding is faster.

Joint conclusion – why expressiveness does not automatically win

First-order logic is more **expressive**: 12 general rules say what 217,020 ground rules say, they state regulations once, a change of the attendance threshold is one edit instead of s·c edits, and new students need no new rules. That is an advantage in **knowledge size, memory and maintenance**, and it grows with s and c. But expressiveness is paid for at inference time: every use of a general rule requires unification and substitution, which cost 15–58 unifications per query here, and the answer is the same as the one the propositional engine finds with about 8 lookups – 27–62× faster per query at every size in our measurements. The cost does not vanish when the KB is large, because it is paid per inference step, not per KB; what grows with the KB is only the *size of the propositional alternative*. Hence the preferable encoding depends on how many queries are asked per KB version, on how often the KB changes, and on whether the instance is small enough to ground cheaply. Greater expressiveness gives better engineering and scaling of the **representation**, not automatically better **reasoning speed**.

TASK 4 – The Reality of Unification and Lifted Inference (7 Marks)

The responsibilities in this task are reversed compared with Task 3.

Shared Group Work: Implement UNIFY (with substitution composition and an optional occurs check), lifted Modus Ponens, and propositionalisation of a small FOL KB (instantiating universal sentences with all ground terms).

Student B – Logical Design and Correctness

- Derive the most general unifier by hand for five pairs of terms that you design, including a pair where the occurs check matters and one where standardising apart is essential
- Explain why lifted inference is sound and why it can avoid generating ground instances

Student A – Experimental and Engineering Analysis

- Measure the number of ground clauses as the number of constants grows (k = 3 to 30) and as predicate arity grows
- Introduce a function symbol and show how the number of ground terms grows with the term depth bound
- Compare inference steps and time for lifted versus propositional inference on the same query

Joint Engineering Decision

Under what conditions would you propositionalize a knowledge base despite the availability of lifted inference? State the conditions using your own evidence.

Evidence Expected: Unifier code and tests, hand derivations, ground-clause growth tables and plots, a lifted-versus-propositional comparison and the engineering decision.

ANSWER – TASK 4: Unification and Lifted Inference

Code: `fol.py` (UNIFY, `compose`, `rename` for standardising apart, optional occurs check, lifted Modus Ponens inside the chaining engines) and `t4_unify.py` (hand-derived test pairs, propositionalisation, growth experiments, lifted-versus-propositional comparison). Raw results: `data/t4.json`.

Shared group work – implementation

```
def unify(x, y, th, occurs_check=True):
    """returns extended substitution (new dict) or None.  MGU by Robinson-style recursion."""
    STATS['unifications'] += 1
    return _unify(x, y, th, occurs_check)

def _unify(x, y, th, oc):
    if th is None: return None
    x = walk(x, th); y = walk(y, th)
    if x == y and type(x) == type(y): return th
    if is_var(x): return _bind(x, y, th, oc)
    if is_var(y): return _bind(y, x, th, oc)
    if isinstance(x, tuple) and isinstance(y, tuple):
        if x[0] != y[0] or len(x) != len(y): return None
        for a, b in zip(x[1:], y[1:]):
            th = _unify(a, b, th, oc)
            if th is None: return None
        return th
    return None

def _bind(v, t, th, oc):
    if oc and occurs(v, t, th): return None
    th2 = dict(th); th2[v] = t; return th2
def compose(s1, s2):
    """(s1 ∘ s2): applying the result equals applying s1 then s2"""
    out = {v: subst(t, s2) for v, t in s1.items()}
    for v, t in s2.items():
        if v not in out: out[v] = t
    return {v: t for v, t in out.items() if v != t}
```

- **UNIFY** returns an extended substitution (a new dictionary) or failure; `occurs_check=True/False` switches the occurs check. Variables are strings starting with "?"; compound terms and atoms are tuples (functor, args…).
- **Substitution composition** `compose(s1, s2)` gives the substitution that has the effect of applying s1 and then s2; a test asserts that Subst(Subst(t, s1), s2) = Subst(t, compose(s1, s2)).
- **Standardising apart** `rename` gives every use of a rule or clause fresh variable names (?x → ?x_17).
- **Lifted Modus Ponens:** from facts p1′,…,pn′ and a rule p1 ∧ … ∧ pn ⇒ q, find θ that unifies each pi with pi′ (UNIFY applied conjunct by conjunct, threading θ) and infer Subst(θ, q). The backward and forward chainers of Task 6 are built on exactly this step.

- **Propositionalisation** instantiates every universally quantified rule with every tuple of ground terms (`propositionalise`), producing ground clauses.

Student B – logical design and correctness

(i) Five term pairs – most general unifiers derived by hand and checked by the program

pair	terms	derivation by hand	hand result	program (occurs check on)
P1	Knows(John, x) and Knows(y, Mother(y))	Unify the arguments left to right. John with y → {y/John}. x with Mother(y): apply the substitution so far, Mother(John), → x/Mother(John). Composition gives θ = {y/John, x/Mother(John)}.	θ = {y/John, x/Mother(John)}	{?y/John, ?x/Mother(John)}
P2 (occurs check)	P(x, f(x)) and P(f(y), y)	x with f(y) → {x/f(y)}. Second arguments: f(x) becomes f(f(y)) and must unify with y. y would have to equal a term that contains y – y occurs in f(f(y)) – so the occurs check fails: **no unifier**. Without the occurs check the algorithm binds y/f(f(y)): a cyclic binding that stands for an infinite term and makes substitution loop.	fails	no unifier (without occurs check: cyclic binding, substitution does not terminate)
P3 (standardising apart)	Knows(John, x) and Knows(x, Elizabeth)	Taken literally, x/John is forced by the first argument and then John must equal Elizabeth: fail. But the two x's are different universally quantified variables of different sentences. Rename the second: Knows(x1, Elizabeth). Now John with x1 → {x1/John}, x with Elizabeth → {x/Elizabeth}.	fails as written; after renaming θ = {x1/John, x/Elizabeth}	no unifier
P4	Q(y, G(A, B)) and Q(G(x, x), y)	y with G(x,x) → {y/G(x,x)}. Then G(A,B) with y, i.e. with G(x,x): x with A → {x/A}, then x (now A) with B: A ≠ B, two different constants: **no unifier**.	fails	no unifier
P5	R(f(x), g(y, z)) and R(f(h(w)), g(w, k))	f(x) with f(h(w)): x/h(w). y with w: y/w. z with k: z/k. No conflicts.	θ = {x/h(w), y/w, z/k}	{?x/h(?w), ?y/?w, ?z/k}

All five hand results agree with the program. For P3 the program confirms the standardised-apart version gives {?x_1/John, ?x/Elizabeth}. For P2 the program with the occurs check switched off returns a substitution in which y is bound to a term containing y; applying it raises a RecursionError – demonstrating why the check is needed for soundness (an "unifier" that does not make the two terms equal is produced otherwise). Which of the five tests would now give a wrong answer without the occurs check? Only P2.

(ii) Why lifted inference is sound, and why it avoids ground instances

Soundness. Lifted Modus Ponens is Universal Instantiation followed by ordinary Modus Ponens. Suppose M is a model of the KB. Because the rule $\forall \bar{x}$ (p1 ∧ … ∧ pn ⇒ q) is true in M, every instance of it is true in M, in particular the instance Subst(θ, p1 ∧ … ∧ pn ⇒ q). The facts pi' are true in M and, since Subst(θ, pi') = Subst(θ, pi), so are their instances Subst(θ, pi) (a fact with variables is universally quantified, so all its instances hold). Modus Ponens on the instances gives Subst(θ, q), which therefore holds in M. The only requirement on θ is that it makes the premises equal – the unifier is merely a way of choosing which instance to use.

Why it avoids ground instances. Propositionalisation instantiates a rule with *all* k^v tuples of constants (v = number of variables) before any reasoning starts. Lifted inference instantiates only the variables that the goal or the matching facts require, and the most general unifier keeps the others free: the conclusion Subst(θ, q) is the most general conclusion, covering every ground instance that a less general unifier could

have produced, so one lifted step stands for many ground steps. Instances that no fact or query ever touches are never created.

Student A – experimental and engineering analysis

(i) Ground clauses as the number of constants grows (k = 3 … 30)

KB: R1 King(x) ∧ Greedy(x) ⇒ Evil(x) (1 variable); R2 Parent(x,y) ⇒ Anc(x,y) (2 variables); R3 Anc(x,y) ∧ Anc(y,z) ⇒ Anc(x,z) (3 variables). Propositionalisation instantiates R1 with k, R2 with k^2 and R3 with k^3 tuples:

constants k	ground clauses (measured)	$k + k^2 + k^3$
3	39	39
5	155	155
10	1,110	1,110
15	3,615	3,615
20	8,420	8,420
25	16,275	16,275
30	27,930	27,930

The measured count equals $k + k^2 + k^3$ for every k: **cubic growth** in the number of constants, dominated by the rule with the most variables (at k = 30: 27,930 ground clauses from a KB of three sentences).

(ii) Growth with predicate arity

arity a	ground clauses, k = 3	k = 5	k = 10
1	3	5	10
2	9	25	100
3	27	125	1000
4	81	625	10000
5	243	3125	100000

For the single rule P(x1…xa) ⇒ Q(x1…xa) the number of ground clauses is **k^a**: with ten constants, arity 5 already gives 100,000 clauses. Growth is exponential in the arity (number of variables per rule) and polynomial in the number of constants.

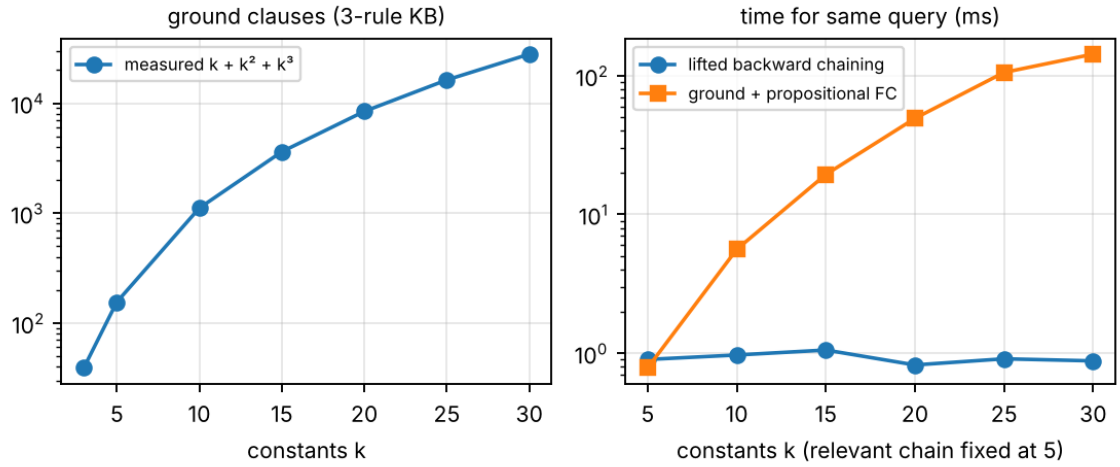


Figure 4.1 – Left: ground clauses against k (log scale). Right: time for one query as irrelevant constants are added.

(iii) Function symbols and the term-depth bound

constants k	function symbols	depth 0	depth 1	depth 2	depth 3	depth 4
1	one unary f	1	2	3	4	5
1	one binary g	1	2	5	26	677
2	one unary f	2	4	6	8	10
2	one binary g	2	6	38	1,446	—
3	one unary f	3	6	9	12	15
3	one binary g	3	12	147	—	—

The Herbrand universe is infinite as soon as there is a function symbol, so propositionalisation needs a depth bound. With one unary function f the number of ground terms with nesting ≤ d is k·(d+1) – linear (k = 2: 2, 4, 6, 8, 10). With one **binary** function g the number of terms obeys $T(d) = T(d-1)^2 + k$ – doubly exponential: for k = 1 the counts are 1, 2, 5, 26, 677, and for k = 2 they are 2, 6, 38, 1,446 and then about 2.1 million (not generated; "—"). Every ground term must then be substituted into every variable of every rule, so a depth bound of only 3 or 4 already makes the propositional KB unmanageable.

(iv) Lifted versus propositional inference on the same query

Query: Anc(c0, c_end) over the KB R2, R3 and Parent facts forming a chain. Lifted = backward chaining with unification (tabled so that the recursive rule R3 terminates) and lifted forward chaining; propositional = ground all rules over the k constants, then counter-based propositional forward chaining (PL-FC) that stops when the query is derived. "steps" = unifications (lifted) or premise-counter decrements (propositional).

Experiment 1 – all k constants lie on the relevant Parent chain

k	lifted BC goals	lifted BC unifications	lifted BC (ms)	lifted FC unifications	lifted FC (ms)	ground clauses	grounding (ms)	PL-FC steps	ground + PL-FC (ms)
3	24	53	0.55	21	0.1	39	0.3	14	0.3
5	40	128	0.87	142	0.4	155	0.7	74	0.8
10	80	508	3.88	2,079	3.8	1,110	4.9	749	5.3
15	120	1263	6.04	10,607	15.7	3,615	17.0	2,774	18.2
20	160	2518	14.56	33,993	50.3	8,420	39.1	6,899	42.1

k	lifted BC goals	lifted BC unifications	lifted BC (ms)	lifted FC unifications	lifted FC (ms)	ground clauses	grounding (ms)	PL-FC steps	ground + PL-FC (ms)
25	200	4398	18.11	82,638	118.5	16,275	78.8	13,874	85.8
30	240	7028	27.87	174,387	242.9	27,930	137.3	24,449	150.0

Experiment 2 – the relevant chain has 5 constants; the other k − 5 constants are irrelevant

k	lifted BC goals	lifted BC unifications	lifted BC (ms)	ground clauses	grounding (ms)	PL-FC steps	ground + PL-FC (ms)	propositional ÷ lifted
5	40	128	0.90	155	0.7	74	0.8	1×
10	40	128	0.97	1,110	5.2	144	5.6	6×
15	40	128	1.05	3,615	18.4	214	19.4	18×
20	40	128	0.82	8,420	45.5	284	49.2	60×
25	40	128	0.91	16,275	98.9	354	105.5	116×
30	40	128	0.88	27,930	134.0	424	142.8	162×

Reading the results. (1) Experiment 2 is the case lifting was invented for: the lifted query always costs 40 goals and 128 unifications (≈ 0.9 ms), independent of k, while the propositional route must create $k + k^2 + k^3$ clauses first – 137 ms at k = 30, about 150 × longer – even though only 11 clauses ever fire. (2) In experiment 1, where every constant is relevant, lifted backward chaining still wins in time (27 ms against 138 ms at k = 30; 7,028 unifications against 27,930 clauses grounded and 24,449 counter decrements), but **lifted forward chaining loses** (238 ms, 176,403 unifications): one unification costs about 2.5 µs in our implementation against 28 ns for a symbol lookup – roughly 90 times more per elementary step – so an engine that touches nearly all instances anyway cannot recoup the overhead. (3) Grounding cost is paid before the first inference and again whenever the constants change.

Joint engineering decision – when to propositionalise despite lifted inference

Our evidence suggests propositionalising when **all** of the following hold; any one failing argues against it.

1. **Finite, small domain and low arity, no function symbols** (or a small depth bound). Ground size is k^v per rule: 1,000 clauses at k = 10, v = 3 but 100,000 at v = 5, and with one binary function symbol the term universe explodes (1,446 terms at depth 3 for k = 2).
2. **Most ground instances will actually be used** (dense relevance, experiment 1) or **many queries are asked against an unchanged KB**, so that the one-off grounding cost is amortised and the 90× cheaper elementary steps of propositional inference pay off (PL-FC: 10.9 ms against 238 ms for lifted FC at k = 30).
3. **A propositional tool is needed**: SAT solving, model counting, enumeration of all models, or the model checker of Task 2 – these have no lifted counterpart of comparable maturity.
4. **The KB is static** or changes rarely, so re-grounding is not a recurring cost.

Conversely, keep the KB lifted when the domain is large or open-ended, the arity is high, function symbols occur, or queries are goal-directed and touch few constants (experiment 2: 0.9 ms against 137 ms and a gap that widens like k^3).

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic (6 Marks)

In Task 2, propositional resolution was applied to a policy KB. Review that solution before attempting

this task.

Shared Group Work: Using the following English statements, design a first-order resolution prover:

1. Every employee who handled a restricted document is authorised.
2. Every document containing patient data is restricted.
3. Some employee handled every document that was audited.
4. Report R1 contains patient data and R1 was audited.

Convert the statements to FOL and CNF (with Skolemisation), implement resolution using your Task 4 unifier, and prove that an authorised employee exists.

Student A – Conversion and Proof Design: Show every conversion step, justify the Skolem function or constant, draw the proof tree and implement one resolution strategy of your choice (unit preference, set of-support or input resolution).

Student B – Comparative Evidence: Record clauses generated, resolution steps, proof length and unifications with and without the strategy. Pose a second query that is **not** entailed and report the behaviour of your prover under a step cap. Compare with the propositional prover of Task 2 on an equivalent small domain.

Cross-Understanding Requirement

• Student B must explain why Skolemisation preserves satisfiability but not logical equivalence • Student A must explain what the experimental evidence shows about the cost of lifting resolution to first-order logic, and what the non-entailed query shows about the decidability of first-order entailment

Evidence Expected: Conversion steps, proof tree, prover code, strategy comparison, behaviour on the non-entailed query and responses to the cross-understanding questions.

ANSWER – TASK 5: Upgrading Resolution from Propositional to First-Order Logic

Code: `t5_fol_resolution.py` (FOL → CNF converter with Skolemisation, first-order resolution prover built on the Task 4 unifier `fol.unify`, three agenda strategies). Raw results: `data/t5.json`. Review of Task 2: the propositional prover of Task 2 (`prop.resolution`) is reused below for the comparison on a small domain.

Shared group work – formalisation

English statement	First-order logic
1. Every employee who handled a restricted document is authorised.	∀x ∀d (Emp(x) ∧ Doc(d) ∧ Restricted(d) ∧ Handled(x,d) ⇒ Auth(x))
2. Every document containing patient data is restricted.	∀d (Doc(d) ∧ Patient(d) ⇒ Restricted(d))
3. Some employee handled every document that was audited.	∃e (Emp(e) ∧ ∀d ((Doc(d) ∧ Audited(d)) ⇒ Handled(e,d)))
4. Report R1 contains patient data and R1 was audited.	Doc(R1) ∧ Patient(R1) ∧ Audited(R1) (R1 is a report, hence a document – stated explicitly because statement 1 needs Doc(R1))
Query: an authorised employee exists	∃x (Emp(x) ∧ Auth(x)) negated: ∀x ¬(Emp(x) ∧ Auth(x))

Student A – conversion and proof design

(i) Every conversion step (output of the program's converter)

Statement 1

step	formula after the step
1. eliminate ⇒	∀x ∀d (¬(((Emp(?x) ∧ Doc(?d)) ∧ Restricted(?d)) ∧ Handled(?x, ?d)) ∨ Auth(?x))
2. move ¬ inwards (NNF)	∀x ∀d ((((¬Emp(?x) ∨ ¬Doc(?d)) ∨ ¬Restricted(?d)) ∨ ¬Handled(?x, ?d)) ∨ Auth(?x))
3. standardise variables apart	∀x1 ∀d2 ((((¬Emp(?x1) ∨ ¬Doc(?d2)) ∨ ¬Restricted(?d2)) ∨ ¬Handled(?x1, ?d2)) ∨ Auth(?x1))
4. Skolemise (remove ∃)	∀x1 ∀d2 ((((¬Emp(?x1) ∨ ¬Doc(?d2)) ∨ ¬Restricted(?d2)) ∨ ¬Handled(?x1, ?d2)) ∨ Auth(?x1))
5. drop ∀ (all variables universal)	(((¬Emp(?x1) ∨ ¬Doc(?d2)) ∨ ¬Restricted(?d2)) ∨ ¬Handled(?x1, ?d2)) ∨ Auth(?x1)
6. distribute ∨ over ∧	(¬Emp(?x1) ∨ ¬Doc(?d2) ∨ ¬Restricted(?d2) ∨ ¬Handled(?x1, ?d2) ∨ Auth(?x1))

Statement 2

step	formula after the step
1. eliminate ⇒	∀d (¬(Doc(?d) ∧ Patient(?d)) ∨ Restricted(?d))
2. move ¬ inwards (NNF)	∀d ((¬Doc(?d) ∨ ¬Patient(?d)) ∨ Restricted(?d))
3. standardise variables apart	∀d3 ((¬Doc(?d3) ∨ ¬Patient(?d3)) ∨ Restricted(?d3))
4. Skolemise (remove ∃)	∀d3 ((¬Doc(?d3) ∨ ¬Patient(?d3)) ∨ Restricted(?d3))
5. drop ∀ (all variables universal)	(¬Doc(?d3) ∨ ¬Patient(?d3)) ∨ Restricted(?d3)
6. distribute ∨ over ∧	(¬Doc(?d3) ∨ ¬Patient(?d3) ∨ Restricted(?d3))

Statement 3 – Skolem symbol introduced: e4 → sk1 (constant)

step	formula after the step
1. eliminate ⇒	∃e (Emp(?e) ∧ ∀d (¬(Doc(?d) ∧ Audited(?d)) ∨ Handled(?e, ?d)))
2. move ¬ inwards (NNF)	∃e (Emp(?e) ∧ ∀d ((¬Doc(?d) ∨ ¬Audited(?d)) ∨ Handled(?e, ?d)))
3. standardise variables apart	∃e4 (Emp(?e4) ∧ ∀d5 ((¬Doc(?d5) ∨ ¬Audited(?d5)) ∨ Handled(?e4, ?d5)))
4. Skolemise (remove ∃)	Emp(sk1) ∧ ∀d5 ((¬Doc(?d5) ∨ ¬Audited(?d5)) ∨ Handled(sk1, ?d5))
5. drop ∀ (all variables universal)	Emp(sk1) ∧ ((¬Doc(?d5) ∨ ¬Audited(?d5)) ∨ Handled(sk1, ?d5))
6. distribute ∨ over ∧	(Emp(sk1)) ∧ (¬Doc(?d5) ∨ ¬Audited(?d5) ∨ Handled(sk1, ?d5))

Statement 4

step	formula after the step
1. eliminate ⇒	(Doc(R1) ∧ Patient(R1)) ∧ Audited(R1)
2. move ¬ inwards (NNF)	(Doc(R1) ∧ Patient(R1)) ∧ Audited(R1)
3. standardise variables apart	(Doc(R1) ∧ Patient(R1)) ∧ Audited(R1)
4. Skolemise (remove ∃)	(Doc(R1) ∧ Patient(R1)) ∧ Audited(R1)
5. drop ∀ (all variables universal)	(Doc(R1) ∧ Patient(R1)) ∧ Audited(R1)
6. distribute ∨ over ∧	(Doc(R1)) ∧ (Patient(R1)) ∧ (Audited(R1))

Negated query

step	formula after the step
1. eliminate ⇒	¬∃x (Emp(?x) ∧ Auth(?x))
2. move ¬ inwards (NNF)	∀x (¬Emp(?x) ∨ ¬Auth(?x))
3. standardise variables apart	∀x6 (¬Emp(?x6) ∨ ¬Auth(?x6))
4. Skolemise (remove ∃)	∀x6 (¬Emp(?x6) ∨ ¬Auth(?x6))
5. drop ∀ (all variables universal)	¬Emp(?x6) ∨ ¬Auth(?x6)
6. distribute ∨ over ∧	(¬Emp(?x6) ∨ ¬Auth(?x6))

The CNF clause set (8 clauses; variables are standardised apart by numbering):

```
C0: ¬Emp(?x1) ∨ ¬Doc(?d2) ∨ ¬Restricted(?d2) ∨ ¬Handled(?x1, ?d2) ∨ Auth(?x1)
C1: ¬Doc(?d3) ∨ ¬Patient(?d3) ∨ Restricted(?d3)
C2: Emp(sk1)
C3: ¬Doc(?d5) ∨ ¬Audited(?d5) ∨ Handled(sk1, ?d5)
C4: Doc(R1)
C5: Patient(R1)
C6: Audited(R1)
C7: ¬Emp(?x6) ∨ ¬Auth(?x6)
```

(ii) Justification of the Skolem constant

In statement 3 the existential quantifier ∃e is the **outermost** quantifier – it is not inside the scope of any universal quantifier – so the employee does not depend on any other variable and the Skolem function has no arguments: it is a **constant** `sk1` ("the employee who handled every audited document"). The inner ∀d stays universal. Had the sentence meant ∀d ∃e (a possibly different employee for every audited document) the Skolem term would be a function sk(d) of the universally quantified variable d. The proof below works under either reading: with sk(R1) in place of sk1 every step is identical, because only the document R1 is ever involved.

(iii) The prover and the strategy implemented

```
    def resolvents(self, i, j):
        c1 = self.cl[i]; c2 = clause_rename(self.cl[j])
        for a, l1 in enumerate(c1):
            for b, l2 in enumerate(c2):
                if l1[0] == l2[0] or l1[1][0] != l2[1][0]: continue
                self.stats['unifications'] += 1
                th = _unify(l1[1], l2[1], {}, self.oc)
                if th is None: continue
                new = [(l[0], subst(l[1], th)) for k, l in enumerate(c1) if k != a] + \
                      [(l[0], subst(l[1], th)) for k, l in enumerate(c2) if k != b]
                yield new, (l1, th)
```

The prover is a **given-clause saturation loop**: pick a clause from the agenda, resolve it with every processed clause (renaming the partner apart, unifying each pair of complementary literals with the Task 4 `unify`, occurs check on), discard tautologies and variant duplicates, and stop when the empty clause is derived, the agenda is empty (saturation) or the step cap is reached. Three agenda strategies were implemented: **none** (all clauses are equal, first-in-first-out), **set of support (SoS)** – only clauses descending from the negated query (the set of support) may be selected, the other clauses serve only as partners – and **unit preference** (shortest clause first). SoS is the strategy of choice for the answer below.

(iv) The proof (set-of-support strategy, 7 resolution steps) and its tree

id	clause	origin	literal resolved and unifier
0	¬Emp(?x1) ∨ ¬Doc(?d2) ∨ ¬Restricted(?d2) ∨ ¬Handled(?x1, ?d2) ∨ Auth(?x1)	input	
1	¬Doc(?d3) ∨ ¬Patient(?d3) ∨ Restricted(?d3)	input	
2	Emp(sk1)	input	
3	¬Doc(?d5) ∨ ¬Audited(?d5) ∨ Handled(sk1, ?d5)	input	
4	Doc(R1)	input	
5	Patient(R1)	input	
6	Audited(R1)	input	
7	¬Emp(?x6) ∨ ¬Auth(?x6)	input	
8	¬Emp(?x1_2344) ∨ ¬Doc(?d2_2345) ∨ ¬Restricted(?d2_2345) ∨ ¬Handled(?x1_2344, ?d2_2345)	resolve 7 and 0	on ¬Auth(?x6) with θ = {?x6/?x1_2344}
10	¬Emp(?x1_2344) ∨ ¬Doc(?d3_2351) ∨ ¬Handled(?x1_2344, ?d3_2351) ∨ ¬Patient(?d3_2351)	resolve 8 and 1	on ¬Restricted(?d2_2345) with θ = {?d2_2345/?d3_2351}
15	¬Doc(?d3_2351) ∨ ¬Handled(sk1, ?d3_2351) ∨ ¬Patient(?d3_2351)	resolve 10 and 2	on ¬Emp(?x1_2344) with θ = {?x1_2344/sk1}
27	¬Doc(?d5_2421) ∨ ¬Patient(?d5_2421) ∨ ¬Audited(?d5_2421)	resolve 15 and 3	on ¬Handled(sk1, ?d3_2351) with θ = {?d3_2351/?d5_2421}
43	¬Patient(R1) ∨ ¬Audited(R1)	resolve 27 and 4	on ¬Doc(?d5_2421) with θ = {?d5_2421/R1}
53	¬Audited(R1)	resolve 43 and 5	on ¬Patient(R1) with θ = {}
57	□ (empty clause)	resolve 53 and 6	on ¬Audited(R1) with θ = {}

Proof tree (each internal node is the resolvent of its two children; leaves are input clauses; the negated query is the clause ¬Emp(x) ∨ ¬Auth(x)):

```
□ (empty clause)   [resolution]
  └ ¬Audited(R1)   [resolution]
    └ ¬Patient(R1) ∨ ¬Audited(R1)   [resolution]
      └ ¬Doc(?d5_2421) ∨ ¬Patient(?d5_2421) ∨ ¬Audited(?d5_2421)   [resolution]
        └ ¬Doc(?d3_2351) ∨ ¬Handled(sk1, ?d3_2351) ∨ ¬Patient(?d3_2351)   [resolution]
          └ ¬Emp(?x1_2344) ∨ ¬Doc(?d3_2351) ∨ ¬Handled(?x1_2344, ?d3_2351) ∨ ¬Patient(?d3_2351)
[resolution]
            └ ¬Emp(?x1_2344) ∨ ¬Doc(?d2_2345) ∨ ¬Restricted(?d2_2345) ∨ ¬Handled(?x1_2344, ?d2_2345)
[resolution]
              └ ¬Emp(?x6) ∨ ¬Auth(?x6)
              └ ¬Emp(?x1) ∨ ¬Doc(?d2) ∨ ¬Restricted(?d2) ∨ ¬Handled(?x1, ?d2) ∨ Auth(?x1)
            └ ¬Doc(?d3) ∨ ¬Patient(?d3) ∨ Restricted(?d3)
          └ Emp(sk1)
        └ ¬Doc(?d5) ∨ ¬Audited(?d5) ∨ Handled(sk1, ?d5)
      └ Doc(R1)
    └ Patient(R1)
  └ Audited(R1)
```

In words: resolving the negated query with statement 1 gives “no employee handled a restricted document”; statement 2 turns “restricted” into “patient document”; Emp(sk1) fixes the employee to sk1; the instance of statement 3 gives Handled(sk1,d) for audited documents; Doc(R1), Patient(R1), Audited(R1) then collapse the clause to □. The substituted values show the answer: **sk1 is an authorised employee** (the witness of ∃x), via the document R1.

Student B – comparative evidence

(i) With and without the strategy – entailed query

strategy	clauses generated	resolution steps	unifications	proof length	time (ms)
no strategy (FIFO)	96	507	507	10	38.7
unit preference	41	53	53	10	2.9
set of support	49	128	128	7	14.1

Without a strategy the prover performs 507 resolution steps to find a 10-step proof; the set-of-support restriction cuts this to 128 steps (a factor of 4.0), 49 instead of 96 generated clauses, and a shorter proof (7 steps); unit preference needs only 53 steps because the unit facts are resolved first, but its proof contains 10 steps since it derives all the unit consequences. About 80 % of all resolvents are duplicates of clauses already present (410 of 507 without strategy) – a sign that subsumption would help further.

(ii) A second query that is not entailed – behaviour under a step cap

Query Q2: $\exists x\,(\mathrm{Emp}(x) \wedge \mathrm{Auth}(x) \wedge \mathrm{Handled}(x, R9))$ – an authorised employee who handled report R9, about which the KB says nothing. It is not entailed. Two knowledge bases were used: the KB above, and the same KB extended by the axiom "every employee has a manager who is an employee", $\forall x\,(\mathrm{Emp}(x) \Rightarrow \exists m\,(\mathrm{Emp}(m) \wedge \mathrm{Mgr}(m,x)))$, whose Skolemisation introduces the **function symbol** mgr(x): clauses $\neg\mathrm{Emp}(x) \vee \mathrm{Emp}(\mathrm{mgr}(x))$ and $\neg\mathrm{Emp}(x) \vee \mathrm{Mgr}(\mathrm{mgr}(x), x)$.

KB without function symbols

strategy	outcome	clauses generated	resolution steps	unifications	time (ms)
no strategy	no (saturated)	141	846	1087	89
set of support	no (saturated)	94	290	378	35
unit preference	no (saturated)	141	846	1087	73

The set of clauses over a finite set of constants and predicates is finite, so the prover reaches **saturation** (agenda empty) without finding □; it correctly reports "not entailed".

KB with the Skolem function mgr(x)

strategy	step cap	outcome	clauses generated	resolution steps	time (s)
no strategy	500	step cap reached	211	500	0.03
no strategy	2000	step cap reached	558	2001	0.22
no strategy	5000	step cap reached	1145	5000	0.84
set of support	500	step cap reached	206	500	0.10
set of support	2000	step cap reached	746	2000	2.54
set of support	5000	step cap reached	1751	5000	23.46
unit preference	500	aborted: term nesting too deep	495	497	0.00
unit preference	2000	aborted: term nesting too deep	495	497	0.00
unit preference	5000	aborted: term nesting too deep	495	497	0.00

No run terminates with an answer. From Emp(sk1) the clause $\neg\mathrm{Emp}(x) \vee \mathrm{Emp}(\mathrm{mgr}(x))$ produces Emp(mgr(sk1)), Emp(mgr(mgr(sk1))), … – an infinite supply of new clauses with ever deeper terms; and the

set-of-support search, which resolves backwards from the query, also keeps producing new clauses. The step caps (500, 2,000, 5,000) are all exhausted; under unit preference the prover chases the unit chain Emp(mgr^n(sk1)) so quickly that the term nesting exceeded the interpreter's recursion limit after about 330 levels (497 steps). The prover can **neither** report "not entailed" **nor** find a proof. (The entailed query Q1 is still proved on this KB: 5,181 steps without a strategy, 362 with set of support – completeness is not lost, only the termination guarantee for non-entailed queries.)

(iii) Comparison with the propositional prover of Task 2 on an equivalent small domain

Equivalent propositional problem: the clause set is instantiated over a finite domain – the constants sk1 and R1 for the smallest case, then 2 more constants each time – and the Task 2 prover (`prop.resolution`: BFS saturation with tautology and subsumption removal) refutes the ground clauses. Atoms are named Emp_sk1, Handled_sk1_R1, and so on. By Herbrand's theorem the two-constant instance is equivalent to the first-order problem because no other constants occur.

domain size k (constants)	ground clauses	clauses generated	resolvents attempted / steps	proof length	time (ms)
2	14	104	284	10	1.4
4	32	180	438	10	3.0
6	58	281	628	10	6.0
8	92	404	836	10	10.7
any k (first-order, no strategy)	8 clauses	96	507	10	38.7
any k (first-order, set of support)	8 clauses	49	128	7	14.1

The propositional effort grows with the domain (104 → 404 generated clauses as k goes from 2 to 8, and ground clauses grow with k^v), whereas the first-order prover works on the same 8 clauses for every k. On the smallest domain the two are comparable (104 generated clauses and 284 attempted resolutions against 96 / 49 clauses and 507 / 128 steps for the first-order prover without / with set of support – not strictly comparable, because the propositional prover has subsumption and the first-order prover does not), but for $k \geq 4$ the propositional cost keeps growing (180, 281, 404 clauses) while the first-order cost stays fixed: lifting pays when the domain is not tiny, and it is the only possibility when the domain is infinite.

Cross-understanding

Student B – why Skolemisation preserves satisfiability but not logical equivalence. Let KB be a set of sentences and KB' the Skolemised set, which uses the new symbols (sk1, mgr, …) that do not occur in KB. (i) If KB has a model M, extend M by interpreting each new symbol as a witness for the existential it replaces (it exists, since ∃ is true in M, and for a Skolem function we choose a witness for every value of its arguments): the extended structure is a model of KB'. Conversely a model of KB' is, after forgetting the new symbols, a model of KB. So KB is satisfiable iff KB' is – which is all a refutation proof needs, because $KB \wedge \neg Q$ is unsatisfiable iff its Skolem form is. (ii) Equivalence would require the same models, but the two sets live in different signatures: $KB' \models KB$ (a Skolem constant is a witness), but $KB \not\models KB'$. Example: $\exists x\, P(x)$ is satisfied by a structure with domain {1,2} where P holds only for 2 and sk is interpreted as 1 – the structure satisfies $\exists x\, P(x)$ but not P(sk). Skolemisation is a conservative extension, not an equivalence transformation, and that is why we may use it inside refutations but not to rewrite a KB and then read answers off it.

Student A – what the evidence shows about the cost of lifting resolution, and what the non-entailed query shows about decidability. *Cost of lifting:* the first-order prover needs a unification per candidate resolution (507 without a strategy, at ≈ 2.5 μs each in our implementation – about 90 times the cost of a symbol comparison in the propositional prover), and without a strategy it is blind: 96 clauses generated and 507 steps for a proof of 10 steps, with 80 % of resolvents duplicates. But its work is independent of the domain size (the propositional prover's effort grows with k), and set of support cut it by a factor of 4; the hidden cost is the infinite Herbrand universe once Skolem functions appear (5,181 steps without strategy against 362 with SoS even for the entailed query on the extended KB). *Decidability:* the not-entailed query on the KB with mgr(x) neither terminated with "no" nor found a proof under any strategy or cap. This is the behaviour predicted by the theory: first-order entailment is **semi-decidable** – resolution is refutation-complete, so a proof of every entailed query will eventually be found, but there is no procedure that halts on every non-entailed query (Church–Turing). Termination in the finite case was an accident of having no function symbols, not a property of first-order resolution.

TASK 6 – Advanced Inference Mini-Investigations (7 Marks)

Both students investigate a different inference strategy on the same knowledge base (an organisation hierarchy with reports-to relations and approval rules, about 10 definite-clause rules, with 100, 1,000 and 10,000 facts).

Student A – Forward Chaining Investigation

- Implement forward chaining to a fixed point
- Record facts inferred, rule firings, redundant derivations, iterations to fixed point and the percentage of derived facts irrelevant to the query
- Explain what happens to the work when the query concerns only a very small part of the KB

Student B – Backward Chaining Investigation

- Implement backward chaining with substitutions
- Engineer a left-recursive rule on which naive backward chaining never terminates, implement a remedy (loop check, depth bound or memoisation) and evaluate its cost
- Record goals expanded, unifications, maximum depth and repeated subgoals

Verification Questions

- Student B must explain: *Why is the choice of inference direction central to performance on this KB?*
- Student A must explain: *Why can backward chaining remain efficient even when the KB contains many facts irrelevant to the query?*

Evidence Expected: Two working engines, metric tables for the three fact sizes, the non-termination demonstration, the remedy, and correct responses to the verification questions.

ANSWER – TASK 6: Advanced Inference Mini-Investigations (Forward and Backward Chaining)

Code: `fol.py` (`FC` semi-naive forward chaining, `BC` backward chaining with substitutions, loop check and depth bound, `TabledBC` memoised backward chaining) and `t6_chaining.py` (KB generator and experiments). Raw results: `data/t6.json`.

The shared knowledge base: an organisation hierarchy (10 definite-clause rules)

```
R1  Superior(x,y) :- ReportsTo(x,y).
R2  Superior(x,z) :- ReportsTo(x,y), Superior(y,z).                    (right-recursive)
R3  Manager(m) :- ReportsTo(e,m).
R4  Approves(m,r) :- Requests(e,r), ReportsTo(e,m), Small(r).
R5  Approves(m,r) :- Requests(e,r), Medium(r), Superior(e,m), Director(m).
R6  Approves(m,r) :- Requests(e,r), Large(r), Superior(e,m), Exec(m).
R7  Escalates(r,m) :- Requests(e,r), ReportsTo(e,m), Medium(r).
R8  Escalates(r,m2) :- Escalates(r,m), ReportsTo(m,m2).
R9  Decided(r) :- Approves(m,r).
R10 Peer(x,y) :- ReportsTo(x,m), ReportsTo(y,m), x ≠ y.
```

Facts: ReportsTo(e,m) for every employee except the root, Requests(e,r) with a size class Small(r), Medium(r) or Large(r) (60 % / 30 % / 10 %), Exec(m) for the root and its direct reports and Director(m) for managers down to depth 2. The hierarchy is a complete 4-ary tree. Three fact sizes were generated:

nominal size	actual facts	employees	tree depth	query employee (deepest, Large request)
100	101	26	3	e25
1000	1001	326	4	e308
10000	10001	3326	6	e3313

Query used throughout: **Approves(?m, r)**, “who may approve the request r of the deepest employee?” (a Large request, which needs an Exec superior and so requires the longest upward search).

Student A – forward chaining investigation

The engine is a **semi-naive** fixed-point loop: facts new in the previous round form the delta; for every rule and every body atom position, that atom is matched against the delta, atoms to its left against old facts only and atoms to its right against all facts, so each rule instance is enumerated once; new facts are committed at the end of the round. Facts and rules are indexed on every argument position. Each derived fact stores its first justification (rule and premises), which is used to compute the relevant part.

facts	facts inferred	rule firings	redundant derivations	iterations to fixed point	unifications	time (s)	derived facts relevant to the query	derived facts irrelevant
101	200	226	26 (11.5 %)	4	2,432	0.01	1	99.50 %
1001	3511	3989	478 (12.0 %)	6	565,307	0.73	9	99.74 %
10001	43086	47835	4,749 (9.9 %)	8	84,061,675	113.00	13	99.97 %

Table 6.1 – Forward chaining to the fixed point. “Redundant” = a rule firing whose conclusion was already known (or was derived by another firing in the same round). “Relevant” = derived facts that lie in the proof of the answers to Approves(?m, r_leaf) according to the stored justifications.

Derived facts at 10,001 facts by predicate: Superior 18,138, Peer 9,972, Approves 5,581, Escalates 5,238, Decided 3,325, Manager 832. The two largest groups – Peer (all sibling pairs) and Superior (all ancestor pairs) – are irrelevant to the query, yet forward chaining must compute them. Iterations = 4, 6, 8 grow with the depth of the hierarchy (the longest derivation chain), not with the number of facts.

What happens to the work when the query concerns only a small part of the KB? Nothing changes: forward chaining is *data-driven* and does not know about the query, so it computes every consequence of every fact. For the query about one request only 1, 9 and 13 derived facts are needed, while 200, 3,511

and 43,086 are derived – 99.5 %, 99.74 % and 99.97 % of the work is irrelevant, and the proportion approaches 100 % as the KB grows. The cost is linear in the KB (47,835 firings) or worse for rules with joins (the Peer join drives 84 million unifications and 113 s at 10,001 facts), and it is paid again whenever facts change.

Student B – backward chaining investigation

The engine implements SLD backward chaining with substitutions: the goal list is solved left to right; for each goal it first tries matching facts (using the first-argument/any-argument index), then each rule whose head unifies with the goal after standardising the rule apart; a successful unification extends the substitution, the rule body is prepended to the remaining goals, and negative or built-in goals are evaluated when ground. Statistics: goals expanded, unifications, maximum depth and repeated subgoals (a goal that is a variant of one seen before).

facts	goals expanded	unifications	maximum depth	repeated subgoals	time (ms)	answers (approvers)	same answers as FC
101	8	8	1	2	0.21	e6	yes
1001	27	28	6	8	0.42	e0, e4	yes
10001	35	36	8	10	0.54	e0, e2	yes

Table 6.2 – Backward chaining on Approves(?m, r_leaf), right-recursive rule R2. The answers are the approvers; they equal the forward-chaining result.

facts	other query	answers	goals expanded	unifications	max depth
101	Superior(e_leaf,?z)	3	12	14	4
101	Decided(r_leaf)	1	9	9	2
1001	Superior(e_leaf,?z)	4	15	18	5
1001	Decided(r_leaf)	2	28	29	7
10001	Superior(e_leaf,?z)	6	21	26	7
10001	Decided(r_leaf)	2	36	37	9

The left-recursive rule and the non-termination demonstration

R2 is replaced by the left-recursive rule **R2L: Superior(x,z) :- Superior(x,y), ReportsTo(y,z).** For the goal Superior(e, ?z) naive backward chaining first selects the new subgoal Superior(e, ?y), which is a variant of the goal itself, and so on forever.

facts	goals expanded before failure	depth reached	repeated subgoals	outcome
101	4938	989	4933	no termination (recursion depth exhausted: Python stack overflow)
1001	5917	988	5911	no termination (recursion depth exhausted: Python stack overflow)
10001	7866	986	7858	no termination (recursion depth exhausted: Python stack overflow)

The recursion depth grows without bound (≈ 990 levels, where the interpreter's stack is exhausted) and 99.9 % of the goals are repeats of earlier goals. No answer is ever returned although Superior(e, ?z) has 3–6 correct answers.

Remedies and their cost (query Superior(e_leaf, ?z) on R2L)

facts	remedy	answers found / correct	complete?	goals expanded	unifications	time (ms)
101	depth bound 2	1 / 3	NO	5	5	0.08
101	depth bound 3	2 / 3	NO	8	9	0.10
101	depth bound 4	3 / 3	yes	12	14	0.12
101	depth bound 6	3 / 3	yes	22	24	0.26
101	loop check (variant ancestors)	1 / 3	NO	3	3	0.04
101	memoisation / tabling (3 passes)	3 / 3	yes	12	28	0.22
1001	depth bound 3	2 / 4	NO	8	9	0.12
1001	depth bound 4	3 / 4	NO	12	14	0.15
1001	depth bound 5	4 / 4	yes	17	20	0.19
1001	depth bound 7	4 / 4	yes	29	32	0.28
1001	loop check (variant ancestors)	1 / 4	NO	3	3	0.06
1001	memoisation / tabling (4 passes)	4 / 4	yes	18	44	0.26
10001	depth bound 5	4 / 6	NO	17	20	0.27
10001	depth bound 6	5 / 6	NO	23	27	0.30
10001	depth bound 7	6 / 6	yes	30	35	0.33
10001	depth bound 9	6 / 6	yes	46	51	0.50
10001	loop check (variant ancestors)	1 / 6	NO	3	3	0.06
10001	memoisation / tabling (6 passes)	6 / 6	yes	33	85	0.53

Table 6.3 – Remedies for left recursion. The depth bounds shown are (depth − 1), depth, (depth + 1) and (depth + 3) of the hierarchy.

Evaluation. (a) A **depth bound** terminates but is correct only if the bound is at least the depth of the proofs: with a bound one or two below the hierarchy depth it silently returns *fewer answers* (for example 4 of 6 at 10,001 facts), and a bound that is large enough costs 12, 17 and 30 goals at 101, 1,001 and 10,001 facts, and a bound three levels too generous 22, 29 and 46 goals – it needs knowledge of the data that a designer may not have. (b) A **loop check** that prunes a goal that is a variant of one of its ancestors terminates immediately (3 goals) but is **incomplete**: it prunes the recursive call, so only the direct manager is found (1 of 3–6 answers). Termination bought with wrong answers is no remedy for this rule. (c) **Memoisation (tabling)** stores the answers of each variant subgoal; a subgoal already in progress consumes the answers found so far instead of being re-expanded, and the query is re-run until no table changes (3–6 passes). It is complete (all answers), uses 12–33 goals and 28–85 unifications – the same order as the right-recursive program – and its cost is the memory of the tables and the extra passes. Memoisation is therefore the remedy to use.

Comparison of the two engines

facts	FC firings	FC unifications	FC time (s)	BC goals	BC unifications	BC time (ms)	FC ÷ BC time
101	226	2,432	0.01	8	8	0.21	54×
1001	3,989	565,307	0.73	27	28	0.42	1,714×
10001	47,835	84,061,675	113.00	35	36	0.54	209,361×

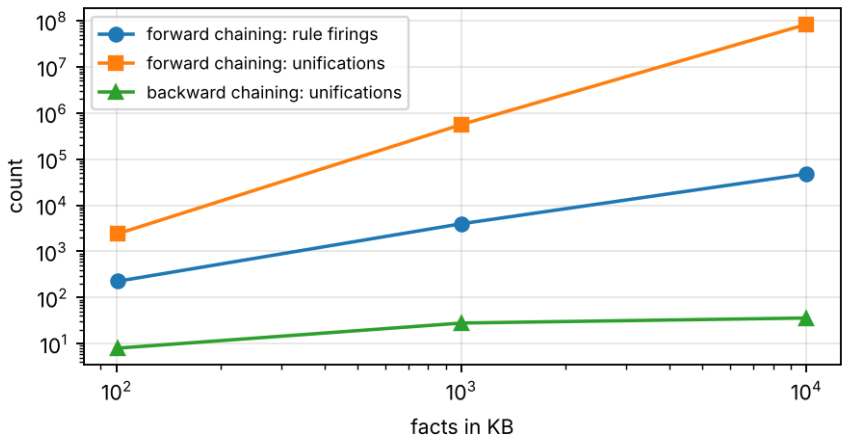


Figure 6.1 – Work done by the two engines for the same query as the KB grows 100-fold (log–log).

Verification questions

Student B – Why is the choice of inference direction central to performance on this KB? The query asks about **one** request. Forward chaining derives every consequence of every fact – at 10,001 facts 43,086 facts, 47,835 firings and 84 million unifications for 13 needed facts – so its cost is a function of the KB. Backward chaining starts from the goal Approves(?m, r17), uses the Requests fact to find the requester, follows ReportsTo upwards, and checks the size class: 35 goals and 36 unifications, and its cost is a function of the proof (tree depth 8), not of the KB. The same inference engine differs by a factor of about 200,000 in time (113 s against 0.54 ms). The direction would be the other way round for a query needing *all* answers ("list every approval for all employees"): then forward chaining computes the answers once, shares work between them (the Superior facts are used by thousands of requests) and avoids re-deriving them for every request, whereas backward chaining repeats the upward search n times.

Student A – Why can backward chaining remain efficient even when the KB contains many facts irrelevant to the query? Because it is goal-directed and the facts are indexed: each subgoal is unified only with the facts that can match it, found through the index on the arguments that are already bound (for example ReportsTo(e3290, ?y) looks at a single fact out of 3,325). Facts about other employees are never touched. The cost is determined by the number of subgoals along the proof – depth × branching of the rules – which grows only with the height of the hierarchy: 8, 27 and 35 goals for 100, 1,001 and 10,001 facts (log-like growth with 100 times more facts). Without an index each goal would scan all facts and BC would also become linear in the KB; and BC becomes inefficient when the proof tree itself is large, when subgoals are repeated (left recursion, shared subproblems – hence tabling) or when many answers are demanded.

DAY-4 DESIGN CHALLENGE

TASK 7 – Design a Knowledge-Based Reasoning Component from First Principles (10 Marks) Problem Context – Hospital Medication-Safety Advisor

Given a patient record (age, allergies, current medicines, conditions) and a newly prescribed drug, the

agent must decide whether the prescription is safe and explain why. The initial knowledge base has about 40 rules. The hospital plans to grow it to 2,000 rules and 50,000 patients.

This problem must be solved collaboratively. Neither student may work independently and merge the answers later.

Stage 1 – Independent Design Proposal – 1 Mark

Before discussing with each other, Student A and Student B must each independently propose: the representation (propositional or FOL), vocabulary, inference method (forward chaining, backward chaining or resolution) and how explanations will be produced. Each initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect.

Stage 2 – Compare the Two Designs – 1 Mark

Identify where the designs agree and differ, their strengths and weaknesses, and which ideas will be retained.

Stage 3 – Joint Final Design – 3 Marks

Produce the final design: representation, vocabulary, inference method, indexing or agenda strategy for scale, explanation mechanism and complete pseudocode.

Stage 4 – Divide the Validation Responsibilities – 3 Marks

- **Student A – Theoretical Validation:** soundness and completeness of the chosen inference, complexity of a query as a function of KB size, and the effect of restricting the KB to definite clauses.
- **Student B – Experimental Validation:** at least 30 test cases including dangerous combinations, KB sizes of 40, 400 and 2,000 rules, metrics (query latency, inference steps, memory, explanation length) and a baseline comparison, with code and outputs as proof.

Stage 5 – Cross-Defence – 1 Mark

Student A explains the experimental evidence supporting the group's design. Student B explains why the theoretical complexity result applies to the group's chosen inference.

Stage 6 – Joint Evaluation – 1 Mark

Why is the problem suitable for a logical agent? What are the limits of the representation (uncertainty, incomplete data)? What should an AI designer check before choosing a knowledge-based approach?

ANSWER – TASK 7: A Knowledge-Based Medication-Safety Advisor, Designed from First Principles

Code: `t7_medsafe.py` (KB, 35 test cases, experiments), `fol.py` (indexed backward chaining and proof trees). Raw results: `data/t7.json`. All drug and clinical rules below are **illustrative teaching data, not clinical advice**.

Stage 1 – Independent design proposals (retained as submitted)

Student A's proposal – propositional logic with forward chaining

- **Representation:** propositional. One atom per fact about the patient and the prescription, e.g. Allergic_penicillin, Takes_warfarin, Age_under_12, Prescribed_amoxicillin; conclusion atoms Unsafe_allergy, Unsafe_interaction_warfarin_ibuprofen, …
- **Vocabulary:** (drug, class, condition) combinations are enumerated as atoms; about 40 rules of the form Allergic_penicillin ∧ Prescribed_amoxicillin ⇒ Unsafe_allergy_amoxicillin.
- **Inference:** forward chaining with an agenda and a premise counter per rule (linear time in the KB), run for each prescription after loading the patient's atoms.
- **Explanations:** every atom stores the rule that first fired it; the explanation is the list of rules fired on the path to the Unsafe atom.

Student B's proposal – first-order Horn clauses with backward chaining

- **Representation:** first-order definite clauses with variables: Allergic(p,c), Takes(p,m), Age(p,a), HasCondition(p,k), InClass(d,c), Interacts(m,d), …; rules with conclusion Unsafe(p,d,reason).
- **Vocabulary:** predicates over patients, drugs, classes and conditions; reasons as terms (allergy(c), interacts(m)).
- **Inference:** backward chaining from the goal Unsafe(patient, drug, ?reason), so that only the rules and facts relevant to the one prescription are touched; "safe" means that no Unsafe proof exists.
- **Explanations:** the proof tree – each rule used and the facts that satisfied it – printed as sentences.

Stage 2 – Comparison of the two designs

	Student A (propositional + forward chaining)	Student B (FOL Horn + backward chaining)
Agree	Rule-based, Horn-like knowledge; no uncertainty; explanations from the inference trace; "no rule fires" means "no known conflict"	(same)
Rule count for 50,000 patients	One copy of the patient-specific atoms per patient, or re-grounding of the rules per run; rules stay ~40 only if each patient is processed separately	40 rules regardless of the number of patients – patients are data
Work per prescription	Forward chaining processes all facts of the patient and fires everything derivable, also for drugs not being prescribed	Goal-directed: only rules for the prescribed drug are tried
Knowledge about classes	Every drug of a class needs its own rule or atom (explosion: classes × drugs)	Class membership as facts (InClass) plus 9 generic rules; a new drug is one fact
Explanation	Trace of fired rules (flat)	Proof tree with rule names and bindings
Weakness	Grows with drugs × classes × conditions; hard to maintain; poor fit for 2,000 rules	Needs unification and indexing; non-termination on left recursion (not present here)
Measured (our baseline, 400 rules)	Forward chaining over 2,000 patients: 1.36 s, 833,327 unifications for 2,000 prescriptions	Backward chaining: ≈ 0.22–0.25 ms and 27–28 unifications per prescription

Ideas retained: Student B's first-order Horn representation and goal-directed backward chaining; Student A's idea of using forward chaining for what does not depend on the prescription – here the offline **materialisation of the drug-class hierarchy** (InClass is closed under SubClass once, before the patients arrive); the premise counters are not needed. Student A's "list of fired rules" is replaced by proof trees.

Stage 3 – Joint final design

Representation and vocabulary

Function-free first-order **definite clauses** (Datalog) with reason terms and negation as failure for the final "safe" decision. Predicate vocabulary:

Predicate	Meaning
Age(p,a), Takes(p,m), Allergic(p,c), HasCondition(p,k), Pregnant(p)	patient record (one fact per item); p is the patient identifier
InClass(d,c), SubClass(c1,c2)	drug d belongs to pharmacological class c; class hierarchy
Interacts(m,d), ClassInteracts(c1,c2), CrossReactive(c1,c2), NoDuplicate(c)	drug–drug, class–class, allergy cross-reactivity, no-duplicate-therapy classes
AvoidPediatric(d), AvoidElderly(d), AvoidPregnancy(d), ContraClass(c,k)	restrictions by age group, pregnancy or condition
Unsafe(p,d,reason)	main derived predicate; reason is a term: allergy(c), cross_allergy(c), interacts(m), class_interaction(m), duplicate(m), pediatric, elderly, pregnancy, condition(k)

Rules: **9 generic rules** G1–G9 (allergy, cross-reactive allergy, drug–drug interaction, class interaction, duplicate therapy, paediatric, elderly, class contraindication, pregnancy) and **drug-specific rules** with the prescribed drug as a constant in the head, for instance Unsafe(p, warfarin, condition(bleeding_disorder)) :- HasCondition(p, bleeding_disorder). 31 such rules complete the 40-rule KB; the 400- and 2,000-rule KBs add further drug-specific rules of the same shape (synthetic drugs). The decision is **Safe(p,d) ≡ ¬∃r Unsafe(p,d,r)** (negation as failure over the complete patient record).

Inference, indexing and explanation

- **Inference method: backward chaining** from Unsafe(p,d,?r), enumerating **all** reasons (a safety check must not stop at the first one).
- **Index for scale:** facts are indexed on every argument position (patient identifier first), so Takes(p,?m) touches only the facts of patient p, not the 50,000 patients; rules are indexed on the **drug argument of the head**, so the goal with drug d meets only the generic rules and the rules for d, not the other ~2,000. Generic rules form the "no constant" bucket.
- **Agenda / subgoal order:** the most selective literal first – the patient fact (a handful per patient) before the knowledge facts.
- **Offline forward chaining:** InClass(d,c2) :- InClass(d,c1), SubClass(c1,c2) is run once to materialise the class hierarchy.
- **Explanation:** the proof tree of each Unsafe fact, printed as sentences (rule name, bindings and the supporting facts).

Pseudocode

```
function CHECK-PRESCRIPTION(kb, p, d):
    reasons <- BACKWARD-ASK(kb, Unsafe(p, d, ?r))        # all answers, no early exit
    if reasons is empty:  return (SAFE, "no known conflict")
    return (UNSAFE, [ EXPLAIN(kb, Unsafe(p, d, r)) for r in reasons ])

function BACKWARD-ASK(kb, goals, theta):                  # SLD resolution with substitutions
    if goals is empty: yield theta; return
    g <- first(goals); rest <- others(goals)
```

```
    if g = not(a):       if BACKWARD-ASK(kb, [a], {}) has no answer: yield from BACKWARD-ASK(kb, rest,
theta)
    elif g is a test (age < 12, ...): if test holds when ground: yield from BACKWARD-ASK(kb, rest,
theta)
    else:
        for fact in FACTS(kb, g):                          # index on the bound arguments (patient first)
            if (t <- UNIFY(g, fact, theta)) != FAIL: yield from BACKWARD-ASK(kb, rest, t)
        for rule in RULES(kb, g):                          # index on the drug argument of the head
            (head, body) <- STANDARDISE-APART(rule)
            if (t <- UNIFY(g, head, theta)) != FAIL:  yield from BACKWARD-ASK(kb, body + rest, t)

function EXPLAIN(kb, ground goal):                         # proof tree -> sentences
    if goal in facts: return "fact: " + goal
    rule <- first rule whose body is provable for goal
    return goal + " because [" + rule.name + "]: " + [ EXPLAIN(kb, b) for b in instantiated body ]

offline, once:  FORWARD-CHAIN({InClass, SubClass}, closure rule)      # class hierarchy
```

Example explanations produced by the system (40-rule KB)

T02 – patient allergic to penicillin, prescribed cephalexin:

```
Unsafe(t02, cephalexin, cross_allergy(penicillin))   [G2 cross-reactive allergy]
  Allergic(t02, penicillin)   [fact]
  CrossReactive(penicillin, cephalosporin)   [fact]
  InClass(cephalexin, cephalosporin)   [fact]
```

T07 – patient on warfarin, prescribed ibuprofen:

```
Unsafe(t07, ibuprofen, class_interaction(warfarin))   [G4 class interaction]
  Takes(t07, warfarin)   [fact]
  InClass(warfarin, anticoagulant)   [fact]
  InClass(ibuprofen, nsaid)   [fact]
  ClassInteracts(anticoagulant, nsaid)   [fact]
```

T27 – pregnant patient taking ciprofloxacin, prescribed warfarin (first of three reasons shown):

```
Unsafe(t27, warfarin, interacts_specific(ciprofloxacin))   [S17 warfarin+ciprofloxacin]
  Takes(t27, ciprofloxacin)   [fact]
```

Stage 4 – Student A: theoretical validation

Soundness. Every step of backward chaining on definite clauses is a resolution step (SLD resolution), and resolution is sound: if the goal Unsafe(p,d,r) is derived, then it is true in every model of the KB, because each use of a rule is universal instantiation followed by Modus Ponens (Task 4). Negation as failure is sound with respect to the closed-world completion of the KB and the stratification here (Unsafe does not depend on itself through negation; Safe is defined only on top of Unsafe). A reported conflict is therefore always justified by an explicit proof – an advisor that raises false alarms from unsound inference would be unusable.

Completeness. SLD resolution with exhaustive search is complete for definite clauses: every ground Unsafe fact that follows from the KB has an SLD proof. Termination is the practical issue: depth-first search can loop on recursive rules (Task 6), but this KB has **no recursion** (proofs have depth ≤ 3: rule, class membership, patient fact), so the search tree is finite and the procedure is complete *and* terminating;

the one recursive rule, the class closure, is run by forward chaining offline. Completeness is relative to the recorded facts: a danger that is not represented (an unrecorded allergy) cannot be found.

Complexity of a query as a function of KB size. Let N be the number of rules (40 … 2,000) and P the number of patients (up to 50,000). With the indexes the work for one prescription is O(r_d · f_p): r_d = 9 generic rules plus the rules whose head names the prescribed drug (a handful – independent of N for a fixed formulary), f_p = facts of the one patient (≈ 5–10) – an average of 18.1 unifications in all three KB sizes. **Independent of N and P** (hash lookups). Without the rule index the cost is Θ(N): every Unsafe rule is tried and most fail at the head unification; measured 47, 407 and 2,007 unifications per query for N = 40, 400, 2,000 – linear. For definite propositional clauses entailment is solvable in time linear in the KB size (Dowling–Gallier) and for function-free first-order Horn clauses in polynomial time in the data (Datalog data complexity); for **general** propositional sentences it is co-NP-complete and for first-order logic only semi-decidable.

Effect of restricting to definite clauses. Gains: polynomial (here essentially constant) query time; one proof per conclusion; a natural proof tree as explanation; no search over models. Losses: no disjunctive or negative information ("allergic to penicillin **or** cephalosporin – unknown which", or an explicitly recorded "no allergy"), no uncertainty, no exceptions except through negation as failure; and negation as failure needs the **closed-world assumption**, so an item missing from the record is treated as false. The restriction is acceptable here because guideline rules are conditional ("if … then unsafe").

Stage 4 – Student B: experimental validation

(i) Test cases (35, including the dangerous combinations)

id	scenario	new drug	expected reasons	reasons found	result	latency (µs)	unif.	explanation lines
T01	direct allergy	amoxicillin	allergy	allergy	PASS	147	18	3
T02	cross-reactive allergy	cephalexin	cross_allergy	cross_allergy	PASS	136	17	4
T03	allergy, unrelated drug	ciprofloxacin	safe	safe	PASS	139	17	–
T04	allergy to parent class (hierarchy closure)	ampicillin	allergy	allergy	PASS	137	16	3
T05	drug-drug interaction	ciprofloxacin	interacts	interacts	PASS	156	20	3
T06	interaction, symmetric direction	warfarin	interacts	interacts, interacts_specific	PASS	172	23	2
T07	class interaction anticoagulant+NSAID	ibuprofen	class_interaction	class_interaction	PASS	154	21	5
T08	class interaction (reverse order)	warfarin	class_interaction	class_interaction	PASS	175	24	5
T09	ACE inhibitor + K-sparing	spironolactone	class_interaction	class_interaction	PASS	162	22	5
T10	nitrate + PDE5	sildenafil	class_interaction	class_interaction	PASS	163	20	5
T11	statin + macrolide	clarithromycin	class_interaction	class_interaction	PASS	153	20	5
T12	benzodiazepine + opioid	codeine	class_interaction	class_interaction	PASS	163	21	5

id	scenario	new drug	expected reasons	reasons found	result	latency (µs)	unif.	explanation lines
T13	duplicate NSAID	naproxen	duplicate	duplicate	PASS	161	22	6
T14	duplicate benzodiazepine	alprazolam	duplicate	duplicate	PASS	165	22	6
T15	different classes, no rule fires	lisinopril	safe	safe	PASS	160	21	–
T16	paediatric restriction	codeine	pediatric	pediatric	PASS	129	14	4
T17	paediatric, permitted drug	amoxicillin	safe	safe	PASS	119	12	–
T18	boundary: age 12 is not paediatric	codeine	safe	safe	PASS	127	13	–
T19	elderly restriction	diazepam	elderly	elderly	PASS	131	14	4
T20	boundary: age 65 is elderly	alprazolam	elderly	elderly	PASS	129	14	4
T21	boundary: age 64 is not elderly	alprazolam	safe	safe	PASS	123	13	–
T22	class contraindication (ulcer + NSAID)	naproxen	condition	condition	PASS	130	15	4
T23	class contraindication (renal + metformin)	metformin	condition	condition	PASS	137	16	4
T24	drug-specific contraindication	warfarin	condition	condition	PASS	146	17	2
T25	pregnancy restriction	warfarin	pregnancy	pregnancy	PASS	135	16	3
T26	pregnancy, permitted drug	amoxicillin	safe	safe	PASS	118	13	–
T27	two reasons at once (pregnancy + interaction)	warfarin	interacts, pregnancy	interacts, interacts_specific, pregnancy	PASS	182	25	2
T28	elderly + ulcer + NSAID	naproxen	condition, elderly	condition, elderly	PASS	136	16	4
T29	empty record, ordinary drug	amoxicillin	safe	safe	PASS	117	12	–
T30	polypharmacy, no conflict	aspirin	safe	safe	PASS	226	37	–
T31	methotrexate + NSAID (specific rule)	ibuprofen	interacts	interacts	PASS	152	20	3
T32	paediatric tetracycline	tetracycline	pediatric	pediatric	PASS	130	14	4
T33	pregnancy + tetracycline	tetracycline	pregnancy	pregnancy	PASS	131	15	3
T34	same drug re-prescribed (not flagged: documented limitation)	ibuprofen	safe	safe	PASS	158	22	–
T35	allergy not recorded (closed-world limitation)	amoxicillin	safe	safe	PASS	116	12	–

Table 7.1 – Test suite, run on the 40-rule KB (all 35 pass; identical verdicts for 400 and 2,000 rules). The oracle requires the right verdict and every expected reason; T06 and T27 also report the same interaction twice (once through the drug-level Interacts fact and once through a drug-specific rule) – a redundancy in the knowledge, harmless for safety but worth cleaning. T34 and T35 document limitations (same-drug re-prescription; an allergy that was never recorded).

Dangerous combinations covered: warfarin + NSAID (T07, T08), nitrate + PDE5 inhibitor (T10), ACE inhibitor + potassium-sparing diuretic (T09), statin + macrolide (T11), benzodiazepine + opioid (T12), methotrexate + NSAID (T31), warfarin + ciprofloxacin (T05, T06), cross-reactive allergy (T02), duplicate therapy (T13, T14), and the age / pregnancy / condition restrictions (T16–T33), plus safe cases and boundary ages (T18, T20, T21).

(ii) KB sizes 40, 400, 2,000 – latency, steps, memory, explanation length

rules	facts (34 patients + static)	correct	mean latency (µs)	max latency (µs)	mean unifications per query	KB memory (KB)	mean explanation length (lines)
40	124	35 / 35	146	226	18.1	77	3.9
400	124	35 / 35	149	234	18.1	261	3.9
2,000	124	35 / 35	152	239	18.1	1013	3.9

Latency rises from 146 to 152 µs while the KB grows 50-fold: the number of unifications per query is **constant (18.1)**, the small increase being the cost of the larger dictionary. Memory grows linearly with the rules (77 KB → 1.0 MB). Explanations average 3.9 lines for unsafe cases, one line per rule application or supporting fact.

(iii) Baselines

rules	indexed: latency (µs)	indexed: unifications	no index: latency (µs)	no index: unifications	slowdown without index
40	146	18.1	286	47	2.0×
400	149	18.1	2005	407	13.4×
2,000	152	18.1	9621	2007	63.4×

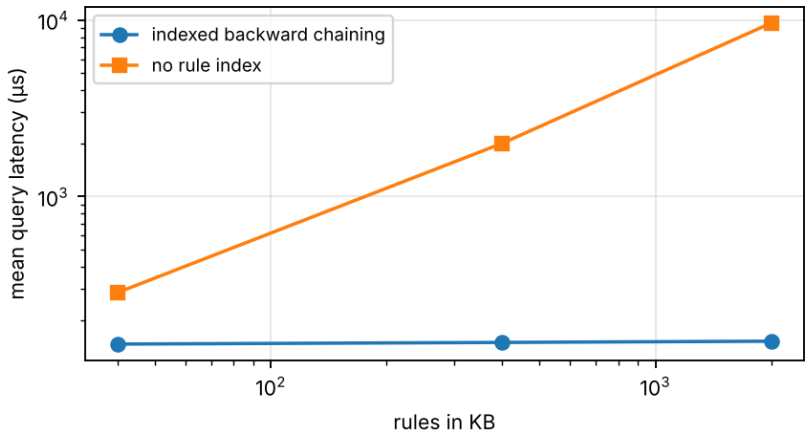


Figure 7.1 – Query latency as the KB grows from 40 to 2,000 rules.

patients	forward chaining over all patients (s)	FC unifications	backward chaining, one prescription (ms)	BC unifications	FC cost per patient (ms)
100	0.07	41,589	0.25	28	0.66
500	0.35	208,557	0.24	27	0.69
2000	1.36	833,327	0.22	27	0.68

Table 7.2 – Baseline (Student A's forward-chaining design, 400 rules, one proposed drug per patient). FC grows linearly with the number of patients; BC stays at about 0.22–0.25 ms per prescription.

(iv) Scaling to 50,000 patients

rules	facts (50,000 patients)	memory (MB)	correct	mean latency (µs)	max latency (µs)
40	137,886	38.2	35 / 35	187	304

rules	facts (50,000 patients)	memory (MB)	correct	mean latency (µs)	max latency (µs)
2,000	137,886	39.1	35 / 35	171	230

With 50,000 patients (137,886 facts) the memory of the KB is about 38–39 MB – dominated by the patient facts, not by the rules – and the latency stays at 152–170 µs per prescription, all 35 verdicts unchanged. The patient-first fact index makes the query independent of the patient population.

Stage 5 – Cross-defence

Student A – the experimental evidence supports the design. (1) 35 / 35 correct verdicts at every size shows that the representation captures the intended rules and that restricting to Horn clauses loses none of the tested situations (the limitations T34/T35 are knowledge-coverage limits, not inference errors). (2) The unifications per query are constant (18.1) for 40, 400 and 2,000 rules, while the same engine without the rule index needs 47 → 407 → 2,007: the index is what makes the 50-fold growth invisible. (3) The patient-first fact index keeps latency at 0.15–0.17 ms when the number of patients rises to 50,000, whereas forward chaining over all patients costs about 0.7 ms per patient and must be repeated for new prescriptions. (4) Proof trees give short explanations (3.9 lines on average) that quote the exact rule and facts.

Student B – why the complexity result applies to the chosen inference. The KB is a set of definite clauses without function symbols, queried by SLD backward chaining. For such programs the proof of a ground goal is a tree whose size is bounded by the depth of the (non-recursive) rule hierarchy and the number of body atoms – here ≤ 3 levels, ≤ 4 atoms – and every atom is resolved against *candidates* found by index lookup. The work is therefore $O(r_d \cdot f_p)$ with r_d and f_p independent of N and P, which is the measured constant 18.1 unifications. If an unindexed list of rules were scanned the same argument gives $\Theta(N)$, exactly the measured linear growth. If the KB contained general (non-Horn) sentences the argument would fail: entailment would need case analysis and could cost exponential time.

Stage 6 – Joint evaluation

Why the problem suits a logical agent. The knowledge is already stated as discrete conditional guidelines ("if the patient takes X and the new drug is in class Y, then unsafe"); every conclusion has to be **justified** to a clinician and auditable – a proof tree is exactly that; the knowledge changes by adding or editing individual rules; the decision is deterministic and repeatable; and the performance requirement (a check at prescription time) is met in 0.15–0.17 ms at 2,000 rules and 50,000 patients.

Limits of the representation. (1) *Incomplete data:* negation as failure treats an unrecorded allergy as no allergy (T35 returns "safe"). The system should therefore answer "no known conflict", flag patients whose record is marked incomplete and could use an explicit RecordComplete(p) predicate to distinguish "verified safe" from "no information". (2) *Uncertainty and severity:* interactions have probabilities and severities, doses and timing matter (the same drug may be safe at 5 mg and unsafe at 50 mg); definite clauses give a yes/no answer and cannot rank risks or weigh benefit against harm. (3) *Exceptions and conflicting guidelines* (non-monotonic reasoning) are only partially covered by negation as failure. (4) *Knowledge acquisition:* correctness is limited by the rule author; the redundancy found (T06, T27) is the kind of defect that creeps in at 2,000 rules and needs tooling and clinical review. (5) *Disjunctive facts* ("allergic to A or B") cannot be stated.

What a designer should check before choosing a knowledge-based approach: whether the knowledge exists as explicit, stable rules (or must be learned from data); whether the facts needed to apply the rules are reliably available and complete; whether answers must be explained and audited; how costly errors of each kind are (a missed danger versus an unnecessary alert); how large and how fast-changing the KB and the data are, and whether an index exists for the queries asked; whether the domain needs uncertainty or numerical reasoning that logic cannot express; and who maintains the knowledge. If most answers are "yes" a logical agent is appropriate; otherwise a hybrid (rules for hard constraints, statistical models for risk) is the better design.

CONTRIBUTION RECORD

Each pair must include this table at the end of the submission, reflecting the actual work completed.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Task 1	Part A: symbol/sentence counts, 2^m, minimal symbol set, soundness and completeness. Part B: inference-rule derivations and step counts vs model checking.	Part A: instrumented checker, runs for n = 2, 3, 4 with and without symbol restriction, three query types. Part B: truth-table soundness proofs, "not derived ≠ false".	Wumpus KB encoding, percept scenarios, truth-table checker, cross-explanation of each other's component.
Task 2	Resolution: CNF by hand, proof tree, clause counters on chains of 10–80, non-entailed query and termination argument.	Model checking: models/time on the same KBs, KB families where it is faster/slower, structural explanation.	Both implementations, the 12-rule access-control policy, test-KB families, joint design question.
Task 3	Design derivation: FOL vocabulary, 14 axioms, growth formula 9sc+sE+s+c.	Crossover investigation: 15 competency queries, defect log, size/time measurements, crossover.	Two encodings, knowledge-engineering process, joint conclusion.
Task 4	Experiments: ground-clause growth (k, arity), function-symbol term growth, lifted vs propositional comparison.	Logic: five hand-derived MGUs, soundness of lifted inference, why ground instances are avoided.	UNIFY, composition, lifted Modus Ponens, propositionalisation, engineering decision.
Task 5	FOL→CNF with every step, Skolem justification, proof tree, set-of-support strategy; explanation	Strategy comparison, non-entailed query under step cap, comparison with propositional prover;	Formalisation of the four

	of cost of lifting and decidability.	explanation of Skolemisation.	statements, first-order resolution prover.
Task 6	Forward-chaining engine, metrics at 100/1,000/10,000 facts, irrelevant-work analysis; answer on BC efficiency.	Backward-chaining engine, left-recursion demonstration, remedies and their cost; answer on direction of inference.	Organisation KB (10 rules) and fact generator.
Task 7	Theoretical validation: soundness, completeness, complexity, definite clauses; cross-defence of the complexity result's counterpart.	Experimental validation: 35 test cases, KB sizes 40/400/2,000, baselines, 50,000 patients; cross-defence of the theory.	Stages 1–3 and 6: independent proposals, comparison, final design, joint evaluation.

MARK DISTRIBUTION – ASSIGNMENT 3

Task	Assessed Area	Marks
Task 1	Logical agents, entailment and reasoning patterns	6
Task 2	Model checking versus resolution	7
Task 3	Knowledge engineering in FOL	7
Task 4	Unification and lifted inference	7
Task 5	First-order resolution	6
Task 6	Forward and backward chaining	7
Task 7	Day-4 Design Challenge	10
	TOTAL	**50**